Dynamic Timetable: A Role-Based Timetable Monitoring System with Automated Timetable Extraction and Natural-Language Queries

submitted in partial fulfillment of the requirement
for the Mini Project (Semester V) of

Third Year Bachelor of Technology
in
Computer Engineering

by

Shivam Mulani	2024300141
Gurshaan Nandrajog	2024300146
Harshvarddhan Waghela	2024300272

Group No. 86

under the guidance of

Prof. Anant Nimkar



Department of Computer Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri-West, Mumbai-400058
University of Mumbai

Academic Year 2026–27

Certificate

This is to certify that the Mini Project entitled "**Dynamic Timetable: A Role-Based Timetable Monitoring System with Automated Timetable Extraction and Natural-Language Queries**" by **Shivam Mulani (2024300141)**, **Gurshaan Nandrajog (2024300146)** and **Harshvarddhan Waghela (2024300272)** (Group No. 86) is a bonafide work carried out under the guidance of **Prof. Anant Nimkar** and is approved for the partial fulfillment of the Mini Project of T.E. Computer Engineering (Semester V), Academic Year 2026–27, of the University of Mumbai.

Certified by

Prof. Anant Nimkar
Project Guide

Dr. Sudhir Dhage
Head of Department

Dr. B. N. Chaudhari
Principal



Department of Computer Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri(W), Mumbai-400058
University of Mumbai
Academic Year 2026–27

Approval Certificate

This is to certify that the Mini Project entitled "**Dynamic Timetable: A Role-Based Timetable Monitoring System with Automated Timetable Extraction and Natural-Language Queries**" by Shivam Mulani, Gurshaan Nandrajog and Harshvarddhan Waghela (Group No. 86) is approved for the partial fulfillment of the Mini Project of T.E. Computer Engineering (Semester V), Academic Year 2026–27, of the University of Mumbai.

Project Guide

(signature)

Name:

Date:

External Examiner

(signature)

Name:

Date:

Head of Department

Principal

Seal of the Institute

Declaration

We wish to state that the work embodied in this report titled "Dynamic Timetable: A Role-Based Timetable Monitoring System with Automated Timetable Extraction and Natural-Language Queries" forms our own contribution to the work carried out under the guidance of Prof. Anant Nimkar at the Sardar Patel Institute of Technology. We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

Shivam Mulani	**Gurshaan Nandrajog**	**Harshvarddhan Waghela**
(signature)	(signature)	(signature)
UID: 2024300141	UID: 2024300146	UID: 2024300272

Date: Place: Mumbai

Acknowledgement

We express our sincere gratitude to our project guide, **Prof. Anant Nimkar**, for the constant guidance, the suggested improvements and the scenario-based challenges that shaped this project, and for reviewing our work at every stage.

We are thankful to **Dr. Sudhir Dhage**, Head of the Department of Computer Engineering, and to **Dr. B. N. Chaudhari**, Principal, Sardar Patel Institute of Technology, for providing the facilities and the encouraging environment in which this mini project was carried out.

We thank the faculty members of the Department of Computer Engineering whose teaching of Software Engineering, Database Management Systems, Operating Systems, Computer Networks, Data Structures and Analysis of Algorithms formed the foundation of this work, and the department office for making the official class timetable available, which we used as the real-world test document for the system.

Finally, we thank our families and friends for their support and for patiently testing the app on their phones.

Shivam Mulani Gurshaan Nandrajog Harshvarddhan Waghela

Abstract

In most colleges the class timetable is published as a multi-page PDF that every student and teacher has to read by hand. Finding the next class, a free room or a teacher's location means scanning dense grids full of abbreviations, lab sub-groups and footnote legends, and every revision of the timetable has to be re-checked manually. Clashes between teachers or rooms are usually discovered only when they happen.

This mini project builds **Dynamic Timetable**, a role-based timetable monitoring system made of a cross-platform mobile and web app (React Native with Expo) and a central back end (Python FastAPI and PostgreSQL). The Admin uploads the official timetable as PDF, Excel, Word or a photo. A background worker extracts every class cell with geometry-aware table parsing and OCR, expands the per-page legends, separates lab sub-groups, validates times and flags anything uncertain for review. Students, professors, the HOD and the Principal then see a dashboard (current class, next class, free periods) and can ask questions in plain English such as "Do I have classes after lunch on Tuesday?", "How many labs do I have this week?" or "Which day is my busiest?". A deterministic compositional planner turns each question into a typed query plan (an operation such as list, count, first/last, total hours or day-wise summary, combined with filters, a date or week, a time window and an ordering), which the server compiles into parameterised queries with the user's role scope added. It never generates SQL from text and cannot be talked into reading other users' data. Institutional and personal timetables are kept strictly separate, and the system automatically detects professor, room and batch clashes.

On the college's real 10-page timetable PDF the system extracted 453 class entries, of which 452 were verified automatically; the one remaining entry has no time printed in the PDF. It also reported exactly the two genuine clashes present in the document. The work is supported by 174 automated back-end tests, 20 mobile unit tests and end-to-end browser tests (24/24 and 21/21 passing). Measured single-user search latency was about 37–47 ms (median) on a 4-core development machine. The system has been deployed as a free online demo. Its limitations are that it has not been load tested on production hardware, push notifications need a store build, and extraction accuracy has not yet been measured against a hand-labelled ground truth.

Keywords: timetable extraction, table recognition, OCR, natural-language interface, intent classification, role-based access control, conflict detection, React Native, FastAPI, PostgreSQL.

Contents

Acknowledgement **iv**

Abstract **v**

Contents **vi**

List of Figures **x**

List of Tables **xi**

List of Abbreviations **xii**

1 Introduction and Problem Definition **1**

1.1 Project Background 1
1.2 Problem Statement 1
1.3 Motivation 2
1.4 Objectives 2
1.5 Scope of the Project 3
1.6 Proposed Solution 3
1.7 Novelty and Originality 4
1.8 Feasibility Analysis 4
1.9 Social Relevance and Practical Applications 5
1.10 Success Criteria and Key Performance Indicators 5

2 Literature Review and Study of Existing Systems **7**

2.1 Study of Existing Systems 7
2.2 Review of Research Papers and Technical Resources 8
2.3 Comparative Analysis 10
2.4 Limitations of Existing Systems 11
2.5 Identified Engineering Gap 12
2.6 Proposed Improvements 12

3 Software Requirements Specification (SRS) **13**

3.1 Purpose and Scope of the SRS 13
3.2 Intended Users and Stakeholders 13
3.3 Overall Product Description 13
3.4 Functional Requirements 14
3.5 Non-Functional Requirements 17
3.6 Hardware and Software Requirements 18

3.7 Constraints and Assumptions . . . 18
3.8 Use Cases and User Stories . . . 19

4 System Architecture and Design 20
4.1 Overall System Architecture . . . 20
4.2 Component and Module Description . . . 21
4.3 Architecture Selection and Justification . . . 22
4.4 Data Flow Diagrams . . . 23
4.5 UML Diagrams . . . 24
4.5.1 Use case diagram . . . 24
4.5.2 Class diagram . . . 25
4.5.3 Sequence diagram: answering a question . . . 25
4.5.4 Activity diagram: publishing a new timetable . . . 26
4.5.5 State machine diagram: timetable processing status . . . 27
4.5.6 Deployment diagram . . . 27
4.6 Database Design . . . 28
4.6.1 ER diagram . . . 28
4.6.2 Relational schema, keys and constraints . . . 28
4.7 Security and Error Handling Design . . . 31

5 Algorithm Design and Data Structures 32
5.1 Identification of Critical Operations . . . 32
5.2 Algorithm Design . . . 32
5.2.1 C1: Time-row resolution by constrained backtracking . . . 32
5.2.2 C2: Cell parsing . . . 34
5.2.3 C3: Conflict detection (group, sort and sweep) . . . 35
5.2.4 C4 and C5: Query planning and interpretation . . . 35
5.2.5 C6: Next class and free rooms . . . 38
5.2.6 C7: Job claiming . . . 38
5.3 Flowcharts . . . 39
5.4 Data Structure Selection . . . 41
5.5 Algorithm Selection and Justification . . . 42
5.6 Time Complexity Analysis . . . 42
5.7 Space Complexity Analysis . . . 43
5.8 Optimisation Opportunities . . . 43

6 Subject-Wise Engineering Concepts and APNI Leap Alignment 45
6.1 Software Engineering . . . 45
6.2 Computer Networks . . . 45

6.3 Database Management Systems . . . 46
6.4 Operating Systems . . . 47
6.5 Data Structures . . . 47
6.6 Design and Analysis of Algorithms . . . 47
6.7 Cloud Computing . . . 48
6.8 Mapping of Subjects to Project Implementation . . . 48

7 Implementation and Module-Wise Coding 50
7.1 Development Environment . . . 50
7.2 Overall Implementation Strategy . . . 50
7.3 Module-Wise Implementation . . . 50
7.4 Front-End Implementation . . . 52
7.5 Back-End Implementation . . . 52
7.6 Database Implementation . . . 53
7.7 Communication and Concurrency Implementation . . . 53
7.8 Git Repository Setup . . . 54
7.9 Integration of Modules . . . 56
7.10 Implementation Challenges and Resolutions . . . 56

8 Scalability, Performance and Resource Management 58
8.1 Scalability Requirements . . . 58
8.2 Workload and Load Model . . . 58
8.3 Resource Utilisation (measured) . . . 59
8.4 Concurrent Processing . . . 59
8.5 Scheduling and Synchronisation Analysis . . . 59
8.6 Performance Measurement . . . 60
8.7 Performance Results (measured) . . . 60
8.8 Bottleneck Identification . . . 61
8.9 Optimisation and Improvement . . . 62
8.10 Cloud Deployment and Resource Utilisation . . . 62
8.11 Scalability Limitations . . . 63

9 Testing and Validation 64
9.1 Testing Strategy . . . 64
9.2 Unit Testing . . . 65
9.3 Integration Testing . . . 65
9.4 Functional Testing . . . 65
9.5 Non-Functional Testing . . . 65
9.6 Boundary and Invalid-Input Testing . . . 66

9.7 Concurrency and Stress Testing . . . 66
9.8 Test Case Specification . . . 66
9.9 Defect Identification and Resolution . . . 69
9.10 Validation Against KPIs . . . 70
9.11 Limitations and Untested Scenarios . . . 70

10 Results, Demonstration and Discussion 72
10.1 Implemented Features . . . 72
10.2 Screenshots and Demonstration Evidence . . . 73
10.3 Functional Results . . . 77
10.4 Performance Results . . . 78
10.5 Comparison with Existing Systems . . . 78
10.6 Achievement of Objectives . . . 79
10.7 Discussion of Findings . . . 79
10.8 Practical Applicability . . . 80

11 Conclusion and Future Scope 81
11.1 Conclusion . . . 81
11.2 Achievement of Objectives . . . 81
11.3 Major Engineering Contributions . . . 81
11.4 Current Limitations . . . 82
11.5 Future Enhancements . . . 82
11.6 Lessons Learned . . . 83

References 84

List of Figures

4.1 Overall system architecture (block diagram) . . . 20
4.2 DFD level 0 (context diagram) . . . 23
4.3 DFD level 1 . . . 23
4.4 Use case diagram . . . 24
4.5 Simplified class diagram of the core domain . . . 25
4.6 Sequence diagram for a natural-language search . . . 25
4.7 Activity diagram: upload, review and activation . . . 26
4.8 State machine of a timetable archive (READY and NEEDS REVIEW may be activated) . . . 27
4.9 Deployment diagram of the free online demo . . . 27
4.10 ER diagram (main entities; personal sections, notifications, tokens and audit omitted) . . . 28
5.1 Flowchart of the extraction pipeline . . . 39
5.2 Flowchart of the query engine . . . 40
8.1 Throughput and latency under increasing concurrent load (measured) . . . 61
10.1 Input: page 1 of the college's real timetable PDF (S.E. Division A), with stacked lab cells and legend . . . 73
10.2 Student screens . . . 74
10.3 Natural-language search (Ask tab) . . . 74
10.4 Combined classes, HOD view, conflicts and personal domain . . . 75
10.5 Administrator workflow . . . 75
10.6 Composed query plans answered from the real timetable (TE-B4 student) . . . 76
10.7 The same app in a laptop browser . . . 77
10.8 Effect of intended-overlap rules: from 542 raw overlaps to the 2 genuine clashes . 78

List of Tables

1.1 Key performance indicators and targets . . . 6
2.1 Summary of reviewed literature and resources . . . 9
2.2 Comparison of existing approaches with the proposed system . . . 11
3.1 Users, stakeholders and their goals . . . 13
3.2 Functional requirements . . . 14
3.3 Non-functional requirements . . . 17
3.4 Hardware and software requirements . . . 18
3.5 User stories and expected system responses . . . 19
4.1 Modules: purpose, inputs, outputs and dependencies . . . 21
4.2 Relational schema of the main tables . . . 28
4.3 Sample entries extracted from the real timetable (page 1, Monday) . . . 30
4.4 Threats and controls . . . 31
5.1 Critical operations of the system . . . 32
5.2 Data structures used and why . . . 41
5.3 Alternatives considered for the key algorithms . . . 42
5.4 Time complexity of critical operations . . . 43
6.1 Mapping of Semester I–IV subjects to the project . . . 48
7.1 Development history (commits on the development branch) . . . 54
7.2 Contribution of group members . . . 55
7.3 Implementation challenges and resolutions . . . 56
8.1 Expected scale of one institute (projected) . . . 58
8.2 Measured resource utilisation . . . 59
8.3 Single-user search latency per question type (50 requests each) . . . 60
8.4 Load test with increasing concurrent clients (4 API workers) . . . 61
8.5 Extraction performance on the real 10-page PDF (median of 5 runs) . . . 61
8.6 Optimisations performed (before/after) . . . 62
9.1 Test suites and results (latest run) . . . 64
9.2 Test case specification . . . 66
9.3 Significant defects found, fixed and re-tested . . . 69
9.4 KPI targets (Chapter 1) vs measured results . . . 70
10.1 Implementation status of features (implemented = code + automated test) . . . 72
10.2 Task effort: manual PDF lookup vs. the app (manual times are estimates) . . . 78
10.3 Objectives mapped to evidence . . . 79

List of Abbreviations

Abbreviation	Full form
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
APNI Leap	Department framework for applying Semester I–IV engineering concepts in projects
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DOCX / XLSX	Office Open XML Word document / spreadsheet
E2E	End-to-End (testing)
ER	Entity–Relationship
FK / PK	Foreign Key / Primary Key
FR / NFR	Functional / Non-Functional Requirement
HOD	Head of Department
HTTP(S)	Hypertext Transfer Protocol (Secure)
IANA	Internet Assigned Numbers Authority (time-zone database)
IDOR	Insecure Direct Object Reference
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
LLC	Liberal Learning Course (co-curricular slot in the college timetable)
NL / NLP	Natural Language / Natural-Language Processing
OCR	Optical Character Recognition
ORM	Object–Relational Mapper
PDF	Portable Document Format
PWA	Progressive Web App
RBAC	Role-Based Access Control
REST	Representational State Transfer

Abbreviation	**Full form**
SDK	Software Development Kit
SQL	Structured Query Language
SRS	Software Requirements Specification
TT	Timetable
UI / UX	User Interface / User Experience
UID	Unique Identification (student identity number)
UML	Unified Modeling Language

Chapter 1: Introduction and Problem Definition

1.1 Project Background

Every semester the timetable committee of an engineering college publishes the class timetable as a multi-page PDF: one grid per division for each of F.E., S.E., T.E., B.E. and M.Tech., with days as rows, time slots as columns and, below each grid, a legend that expands abbreviations such as `DBMS`, `KKD` or `LLC`. A single cell may contain a subject, a teacher code, a room number and, for practical sessions, four stacked lab sub-groups (`A1`–`A4`), each with its own teacher and laboratory. Electives, combined B.E. classes, "Tentative" sections and "W.E.F." (with-effect-from) dates are written in free text around the grid.

Students and teachers read this document by hand many times a day: to find the next lecture, to see whether a lab is in 606 or 607, to locate a teacher, or to look for an empty room for a make-up lecture. When a revised PDF is circulated, everyone has to compare it with the old one manually. Clashes, for example one professor assigned to two rooms at the same time, are usually noticed only when they happen in the corridor. Generic calendar apps do not help, because nobody types 450 classes into a calendar, and timetable-generation software is used only by the committee that creates the timetable, not by the people who follow it.

1.2 Problem Statement

> *Design and implement a system that converts a college's published timetable (PDF, spreadsheet, Word file or photograph) into verified structured data, keeps every version as an auditable archive, detects scheduling clashes, and lets each user (student, professor, HOD, Principal or administrator) get correct, role-appropriate answers to everyday timetable questions on a mobile phone or laptop, including questions typed in plain English, without ever exposing one user's personal data to another.*

The difficulties with the existing approach are:

1. **Unstructured source.** The timetable exists only as a visually formatted document. Merged cells, stacked lab rows, page-specific legends, inconsistent time labels (for example a row printed as "12.15–1.15" without a.m./p.m.) and scanned pages make automatic reading hard.
2. **Manual lookup.** Answering "Where is my next class?" requires locating the right page, row, column and lab sub-group, and expanding the legend codes.
3. **No conflict visibility.** Teacher, room and batch overlaps are not checked mechanically.

4. **Version confusion.** Old and new PDFs circulate together; it is unclear which one is official.
5. **Privacy.** Students may also want to keep a personal timetable (tuition, coaching), which must never be visible to the college administration.

1.3 Motivation

The problem affects every member of the college every working day. A typical department timetable has more than 450 class entries across ten pages, and each of several hundred students and about 50 faculty members consults it several times a day. Even saving one minute per lookup saves hundreds of person-hours per semester. More importantly, an automatically checked timetable removes clashes *before* the semester starts. The project also gave us the opportunity to apply, in one realistic system, almost every core subject of the first four semesters: data structures and algorithms in the parser and query engine, DBMS in the schema and transactions, operating-system concepts in the background worker and concurrency control, computer networks in the client–server API, and software engineering throughout.

1.4 Objectives

The objectives were defined to be specific and measurable:

O1. Extract class entries from the college's real timetable PDF with at least 95% of entries verified automatically, and flag every uncertain value for human review instead of guessing.

O2. Support at least five input formats (PDF, scanned PDF/photo via OCR, XLSX/CSV, DOCX) through one common pipeline.

O3. Provide role-based dashboards (current class, next class, free periods) for five roles: Student, Professor, HOD, Principal and Admin, with server-side enforcement of permissions.

O4. Answer everyday questions typed in English through a deterministic engine that is not limited to a fixed list of questions: at least 25 kinds of question plus free combinations of operations and filters, with clarification instead of guessing when a question is ambiguous.

O5. Detect professor, room and batch clashes automatically and report only genuine clashes.

O6. Keep institutional and personal timetables strictly separate, with complete version history.

O7. Achieve a median search response time below 300 ms for a single user on modest hardware, with no incorrect answers under concurrent use.

O8. Run on Android, iPhone and laptop browsers from one code base, and deploy a free online demo.

O9. Back the system with automated tests (unit, integration, end-to-end) with 100% of tests passing before every commit.

1.5 Scope of the Project

Features included. Timetable upload and extraction (PDF, scanned PDF, JPG/PNG, XLSX, XLS, CSV, DOCX, DOC); validation, review and auditable correction; archive of all versions with one official (primary) timetable; activation notifications; conflict detection; role dashboards; weekly timetable view; natural-language search (compositional query plans); free-room and professor-availability search; personal timetables; academic-year promotion of students; administrator master data (departments, batches, courses, rooms, users, aliases); audit log.

Features outside the scope. Automatic *generation* or optimisation of a timetable (the committee still creates it); attendance; live location tracking of teachers (the system reports only the *scheduled* room); exam timetables; integration with the college ERP; free-form chat with a large language model.

Intended users. Students, professors, Heads of Department, the Principal and the timetable administrator.

Operating environment. Android and iOS phones (as an installable app or a home-screen web app), laptop browsers (Chrome, Edge, Safari, Firefox) and a Linux server or cloud container running Python 3.12+ and PostgreSQL 14+.

Expected scale. One institute: about 2,000–3,000 users, 50–100 batches, 500–1,000 class entries per timetable, a few timetable versions per semester and short bursts of concurrent use around lecture changes.

1.6 Proposed Solution

The proposed system (Figure 4.1) has three parts:

1. **An extraction pipeline** that converts any supported file into a common *grid* model, recognises sections, time rows, lab sub-groups, legends and breaks, validates every entry and assigns it a status: VERIFIED, UNVERIFIED, INCOMPLETE or REJECTED.
2. **A central back end** (FastAPI + PostgreSQL) that stores timetables as archives, enforces role-based access on every request, detects conflicts, and answers questions through a rule-based *compositional query planner*: each question becomes a typed plan (operation, filters,

lab/lecture, date or week, time window, ordering) that is compiled into parameterised queries. Specialised operations (QT01–QT22) cover free rooms, professor free time, archives and similar needs; composed plans run on template QT23.

3. **A cross-platform app** (React Native/Expo) with role dashboards, a weekly view, an "Ask" screen, archives, uploads, review screens and administrator tools.

1.7 Novelty and Originality

- **End-to-end from the real document.** Most student timetable apps expect the timetable to be typed in. This system reads the college's actual PDF, including its specific conventions: stacked lab sub-groups, page-wise legends, the 12:15 "noon" row, combined B.E. electives and co-curricular LLC slots with no teacher or room.
- **Honest uncertainty.** Instead of silently guessing, every entry carries a verification status, the raw source text and its page. Ambiguous a.m./p.m. is never corrected silently.
- **Safe, compositional natural-language queries.** Questions are not limited to a fixed list: independent pieces (count, first/last, busiest day, lab/lecture, before/after a time, this week ...) combine into a typed query plan. The plan contains only values from a closed vocabulary and is compiled with bound parameters and the caller's role scope, so the engine never builds SQL from user text, is immune to prompt or SQL injection by construction, and every answer is reproducible.
- **Strict domain separation.** Institutional and personal data live in separate tables; the code cannot even express a query that unions them.
- **Clash detection with intent awareness.** Parallel lab sub-groups and elective options are recognised as intended overlaps, so only genuine clashes are reported (542 raw overlaps reduced to the 2 genuine clashes in the real PDF).

1.8 Feasibility Analysis

Technical feasibility. All components use mature open-source software: PyMuPDF for PDF geometry, Tesseract for OCR, FastAPI and SQLAlchemy for the API, PostgreSQL for storage and Expo for the app. The team had working knowledge of Python, SQL and JavaScript from earlier semesters. The riskiest part, parsing the real PDF, was prototyped first and proved feasible (Chapter 10).

Economic feasibility. The software stack is free. Development used personal laptops. The online demo runs on Render's free tier (free web service and a free PostgreSQL database), so

the total cost is **Rs. 0**. Production hosting for real students is estimated at about US$7–25 per month (a small always-on instance and a managed database).

Operational feasibility. Users need only a browser or the Expo/installed app. The Admin's workflow (upload, review, activate) mirrors the existing manual process, and the system never changes the official timetable without explicit activation.

Time and resource feasibility. The work was planned over one semester in phases (requirements, back-end foundation, extraction, query engine, app, testing, deployment), which fits the mini-project timeline. Section 7.8 shows the commit history.

1.9 Social Relevance and Practical Applications

- Students, especially first-years and students who switch divisions or electives, can find the right class, lab and room instantly, reducing late arrivals and confusion.
- Faculty can check their own schedule and free periods, and find free rooms for extra lectures.
- The HOD and Principal get a clash report and a view of room usage, which supports better use of classrooms and laboratories, a scarce resource in Indian colleges.
- The archive gives a transparent record of which timetable was official on which date.
- The approach transfers to schools, coaching institutes, hospitals (duty rosters) and any organisation that publishes grid-style schedules as PDFs.
- Accessibility: the plain-English "Ask" screen, large tap targets and dark mode help users who find dense PDF grids hard to read.

1.10 Success Criteria and Key Performance Indicators

Table 1.1 lists the KPIs fixed before testing. Chapter 9 (Section 9.10) compares the measured values with these targets.

Table 1.1: Key performance indicators and targets

KPI	Definition	Target	Measurement method
Extraction verification rate	Verified class entries / all class entries on the real PDF	$\geq 95\%$	Seed run on the real PDF; regression test
False clash reports	Reported conflicts that are not genuine clashes	0	Manual check of every reported conflict against the PDF
Response time	Median server time per NL search, single user	< 300 ms	Timestamped client measurement, 50 requests per query type
Throughput	Searches served per second with 4 API workers	≥ 30 req/s	Multi-threaded load test (Section 8.6)
Reliability	Correct responses / total requests under concurrency	100%	Load test, response status check
Functional correctness	Passed / total automated tests	100%	pytest, Jest, Playwright
Extraction time	Time to extract the 10-page real PDF	< 10 s	Timed pipeline run (median of 5)
Resource usage	Memory of one API worker; database size for one timetable	< 300 MB; < 50 MB	`ps`, `pg_database_size`
Coverage of questions	Specialised operations plus composable plan operations	≥ 25	Registry, plan schema and their tests

Chapter 2: Literature Review and Study of Existing Systems

2.1 Study of Existing Systems

Manual process (current practice). The timetable committee prepares the timetable in a spreadsheet or word processor and circulates a PDF by e-mail, on WhatsApp groups and on notice boards. Users read it by hand. The process costs nothing and everyone understands it, but it supports no search, no personalisation, no clash checking and no version control.

Timetable generation software: FET, UniTime and aSc Timetables. FET [24] is a free, open-source timetable generator. UniTime [25] is an open-source, web-based university course timetabling and student scheduling system. aSc Timetables [26] is a commercial school timetable generator. These tools solve the *construction* problem: they assign events to time slots and rooms under hard and soft constraints, and they export the result as printable grids. They are used by the committee, not by students. They do not read an existing PDF, and their published output is again a static document.

PDF table-extraction tools: Tabula and Camelot. Tabula [27] and Camelot [28] extract tables from text-based PDFs into CSV. They handle ruled tables well, but they return raw cell strings: they do not understand time rows, lab sub-groups, legends, merged two-hour cells or a.m./p.m. ambiguity, and they do not work on scanned images without separate OCR.

General calendar apps: Google Calendar and Outlook. Calendar apps offer reminders and sharing, but every event must be entered by hand or imported from an iCalendar file that the college does not publish. They have no notion of roles, batches, rooms or room availability.

College ERP / LMS timetable modules. ERP products and learning-management systems often include a timetable module, but the data must be keyed in by staff, the modules are usually closed and licensed per student, and they rarely offer free-room search, clash reports or natural-language queries.

Natural-language interfaces built on large language models. Recent assistants translate questions into SQL with large language models (text-to-SQL). They are flexible, but answers are not reproducible, they can produce plausible but wrong queries, and they are vulnerable to prompt-injection attacks that lead to SQL injection [11, 12].

2.2 Review of Research Papers and Technical Resources

Table 2.1 summarises the papers and technical resources reviewed, grouped by theme. The rest of this section discusses the most important ones.

Timetabling. Schaerf's survey [1] classifies school, course and examination timetabling problems and the search techniques used to solve them. Burke and Petrovic [2] review research directions in university timetabling, including constraint-based, meta-heuristic and hybrid methods, and point out the gap between research prototypes and real institutional data. Lewis [3] surveys meta-heuristics for university course timetabling and separates hard constraints (no clashes of teachers, rooms or student groups) from soft ones. These works define the clash model we *check* after publication (professor, room and batch overlap), but they all assume that the timetable is created by the software. None of them addresses reading an already published timetable.

Table recognition and OCR. Zanibbi, Blostein and Cordy [4] survey table recognition and describe the pipeline of table detection, structure recognition (rows, columns, spanning cells) and interpretation. They note that interpretation is domain specific. Kieninger and Dengel [5] (T-Recs) recognise tables from word bounding boxes without ruling lines. We use the same idea in our geometric fallback, which rebuilds columns from whitespace gutters. The ICDAR 2013 table competition [6] established standard evaluation of table location and structure, showing that cell-structure recovery remains error-prone and motivating our per-cell verification status. Smith [7] describes the Tesseract OCR engine (adaptive classifier, line finding and word recognition) used for scanned pages and photographs. PyMuPDF [29] provides text with coordinates and ruled-table detection that preserves merged-cell spans.

Natural-language interfaces to databases. Androutsopoulos, Ritchie and Thanisch [8] review classical natural-language interfaces to databases (NLIDB), including pattern-matching, syntax-based and semantic-grammar systems. They note that a narrow domain allows simple, reliable pattern-based interpretation. Seq2SQL [9] and the Spider benchmark [10] show that neural text-to-SQL can generalise across schemas, but exact-match accuracy on complex queries remains well below 100%, which is unacceptable when a student relies on the answer to reach a lab on time. Greshake et al. [11] and Pedro et al. [12] demonstrate prompt-injection attacks on LLM-integrated applications, including attacks that turn prompts into SQL injection. These findings led us to a closed intent registry with parameterised templates instead of generated SQL.

Time intervals and conflicts. Allen's interval algebra [13] defines the thirteen relations between time intervals. Our clash rule uses the *overlaps/during/equals* family, $a.start < b.end \wedge b.start < a.end$, with start-inclusive, end-exclusive slots, so consecutive lectures do not clash.

Security and architecture. Sandhu et al. [14] define role-based access control (RBAC), which underlies our five roles and their data scopes. JSON Web Tokens are specified in RFC 7519 [18]. Argon2, the winner of the Password Hashing Competition, is a memory-hard function resistant to GPU cracking [15]. The OWASP Top 10 [19] lists broken access control and injection among the most critical web risks, and it guided our IDOR and injection controls. Fielding's REST dissertation [16] motivates the stateless resource-oriented API, and Codd's relational model [17] with normalisation theory [20] underlies the schema. PostgreSQL's `SELECT ... FOR UPDATE SKIP LOCKED` [30] lets several worker processes take jobs from one table without a separate message broker.

Table 2.1: Summary of reviewed literature and resources

#	Work	Contribution	Relevance / limitation for our problem
1	Schaerf, 1999 [1]	Survey of school, course and exam timetabling formulations and algorithms	Defines clash constraints; assumes timetable is generated, not read
2	Burke & Petrovic, 2002 [2]	Research directions in university timetabling	Highlights gap between research and real institutional data
3	Lewis, 2008 [3]	Survey of meta-heuristics for university timetabling; hard vs soft constraints	Hard constraints become our conflict checks
4	Zanibbi et al., 2004 [4]	Survey of table recognition: detection, structure, interpretation	Interpretation is domain specific, so we built a timetable-specific grid parser
5	Kieninger & Dengel, 1998 [5]	T-Recs: tables from word boxes without ruling lines	Basis of our geometric (gutter-based) fallback
6	Göbel et al., 2013 [6]	ICDAR 2013 table competition and metrics	Structure recovery is error-prone, so we assign a status to each entry

#	Work	Contribution	Relevance / limitation for our problem
7	Smith, 2007 [7]	Architecture of the Tesseract OCR engine	OCR for scans/photos; OCR entries capped at UNVERIFIED
8	Androutsopoulos et al., 1995 [8]	Review of NL interfaces to databases	Narrow domain allows reliable rule-based interpretation
9	Zhong et al., 2017 [9]	Seq2SQL: neural text-to-SQL with RL	Flexible but not exact; needs training data
10	Yu et al., 2018 [10]	Spider cross-domain text-to-SQL benchmark	Accuracy on complex queries far below 100%
11	Greshake et al., 2023 [11]	Indirect prompt injection in LLM-integrated apps	Risk of letting an LLM drive data access
12	Pedro et al., 2023 [12]	Prompt-to-SQL injection in LLM web apps	Motivates templates instead of generated SQL
13	Allen, 1983 [13]	Interval algebra for temporal reasoning	Formal basis of the overlap/-clash rule
14	Sandhu et al., 1996 [14]	Role-based access control models	Five roles with data scopes
15	Biryukov et al., 2016 [15]	Argon2 memory-hard password hashing	Password storage
16	Fielding, 2000 [16]	REST architectural style	Stateless JSON API design
17	Codd, 1970 [17]	Relational model of data	Normalised relational schema
18	RFC 7519 [18], OWASP Top 10 [19]	JWT standard; top web security risks	Token format; IDOR and injection controls
19	FET, UniTime, aSc [24–26]	Timetable generators	Construct timetables; no reading, search or role views
20	Tabula, Camelot, PyMuPDF [27–29]	PDF table extraction tools	Raw cells only; no timetable semantics

2.3 Comparative Analysis

Table 2.2 compares the existing approaches on criteria that matter to the users of a published college timetable.

Table 2.2: Comparison of existing approaches with the proposed system

Criterion	**Manual PDF**	**FET / UniTime / aSc**	**Tabula / Camelot**	**Calendar apps**	**ERP module**	**LLM text-to-SQL**	**Proposed**
Reads published PDF/photo	–	No	Partly	No	No	No	**Yes**
Timetable semantics (labs, legends)	Human	Yes	No	No	Yes	Depends	**Yes**
Role-based views	No	Limited	No	No	Yes	No	**Yes (5)**
Plain-English questions	No	No	No	Limited	No	Yes	**Yes (composed plans)**
Reproducible, injection-safe answers	–	–	–	–	Yes	No	**Yes**
Clash detection	No	At creation	No	No	Sometimes	No	**Yes, after publication**
Version archive	Ad hoc	Files	No	No	Sometimes	No	**Yes, audited**
Personal timetable kept private	–	No	No	Yes	No	No	**Yes**
Cost	Free	Free/Paid	Free	Free	Paid	Paid API	**Free (open source)**
Mobile use	PDF viewer	Limited	No	Yes	Varies	Varies	**Android, iOS, web**

2.4 Limitations of Existing Systems

1. Generators (FET, UniTime, aSc) stop at a printable grid; the people who follow the timetable cannot search or personalise it.
2. Generic table extractors do not understand timetable conventions: stacked labs become one garbled string, legends are not applied, and times stay ambiguous.
3. Calendars and ERP modules require manual data entry, which is error-prone and is rarely repeated when the timetable is revised.
4. No existing tool used by our college detects clashes in the *published* timetable.
5. LLM-based interfaces are not reproducible and introduce injection risks [12].

2.5 Identified Engineering Gap

There is no free system that takes the timetable *as the college actually publishes it* and turns it into trustworthy, role-scoped, searchable data with provenance, clash checks and version history, while answering everyday questions in plain English safely. The gap is not a better timetable-construction algorithm. It is the missing link between the published document and its daily users.

2.6 Proposed Improvements

- A timetable-aware extraction pipeline with geometry, OCR and office adapters feeding one grid model, with per-entry provenance and verification status (closes limitations 2 and 3).
- Admin review, correction and activation with an audited archive (closes limitation 3).
- Interval-based clash detection that understands parallel lab groups and elective options (closes limitation 4).
- A compositional NL query planner (typed plans compiled into parameterised queries) with clarification instead of guessing (closes limitation 5).
- Role dashboards and a single code base for Android, iOS and the web (closes limitation 1).

Chapter 3: Software Requirements Specification (SRS)

3.1 Purpose and Scope of the SRS

This SRS defines what the Dynamic Timetable system must do (functional requirements) and how well it must do it (non-functional requirements) for its first release (V1). It is the reference for design (Chapter 4), implementation (Chapter 7) and testing (Chapter 9): every functional requirement has an identifier that appears in the test-case table (Section 9.8). The SRS covers the back-end API, the extraction worker and the cross-platform client; timetable *generation*, attendance and ERP integration are excluded (Section 1.5).

3.2 Intended Users and Stakeholders

Table 3.1: Users, stakeholders and their goals

User / stakeholder	**Goals and permissions**
Student	See own batch and lab-group classes, next class, free periods, free rooms; ask questions; keep a private personal timetable.
Professor	See own teaching schedule and free periods; find rooms; locate colleagues' *scheduled* room.
HOD	Everything a professor can do, plus the department's clash report, floor activity and room usage.
Principal	College-wide read access to dashboards, clashes and room usage.
Admin (timetable coordinator)	Upload, review, correct and activate official timetables; manage master data, users, roles and promotions; read the audit log. Cannot read anyone's personal timetable.
Timetable committee	Benefits from automatic clash detection before publishing.
College management	Transparent version history; better room utilisation.

3.3 Overall Product Description

System environment. A client–server system. Clients are the Expo/React Native app (Android, iOS) and the same app compiled for the web (laptop browsers, home-screen web app). The server is a stateless FastAPI application behind HTTPS, a PostgreSQL database, a background worker for extraction and private file storage for uploaded originals.

Major functions. (1) authentication and role-based authorisation; (2) timetable upload, extraction, validation, review, correction and activation; (3) archives and primary (default) selection per domain; (4) dashboards and weekly views; (5) natural-language search; (6) conflict detection; (7) academic administration (departments, years, batches, courses, rooms, users, promotion); (8) notifications and audit.

Dependencies. PostgreSQL 14+ (tested 16); Tesseract OCR for scans/photos; LibreOffice only for legacy `.xls`/`.doc`; an SMTP server for e-mail (optional; without it the Admin hands out activation codes); Expo push service for push notifications (store builds only).

Operating conditions. Times are stored as local wall-clock times of the institute; "now" is computed in the user's IANA time zone (default `Asia/Kolkata`). Week start, lunch boundary and working hours are configured by the Admin and never assumed.

3.4 Functional Requirements

Table 3.2 lists the functional requirements. For each one it gives the input, the expected processing and output, and the acceptance criterion used in testing.

Table 3.2: Functional requirements

ID	Requirement	Input → processing → output	Acceptance criterion
FR-01	Sign-in with college e-mail and password	E-mail, password → Argon2id verify, account-status check, rate limit → 15-min access JWT + rotating refresh token	Wrong e-mail or password gives the same generic error; suspended account refused
FR-02	Five roles enforced on the server	Request + token → roles loaded from `USER_ROLE` on every request → allowed / 403	No endpoint trusts a client-sent role; role change invalidates old tokens
FR-03	Upload a timetable (PDF, image, XLSX/XLS, CSV, DOCX/DOC)	File → extension allow-list, content-signature check, size/page limits, private storage, SHA-256, job queued → archive record (QUEUED)	Upload never changes the primary timetable; retries with the same request id are idempotent
FR-04	Extract class entries	Stored file → worker: page inspection, table/OCR extraction, sections, time rows, cell parsing, legend expansion, mapping → entries with provenance and status	Real college PDF: $\geq 95\%$ of class entries VERIFIED; nothing guessed silently

ID	Requirement	Input → processing → output	Acceptance criterion
FR-05	Validate and classify the timetable	Entries → checks (missing fields, duplicates, time consistency, overlaps) → READY / NEEDS REVIEW / UNUSABLE / FAILED	Unusable timetables cannot become primary
FR-06	Review and correct entries	Admin edits → before/after snapshot in `ENTRY_CORRECTION`, audit row → corrected entry	Original value always recoverable; entry becomes VERIFIED only by explicit reviewer action
FR-07	Activate the official timetable	Admin action → settings row locked, primary switched, version incremented, users notified → new primary	Concurrent activations serialise; next request sees the new primary
FR-08	Archives and selection	User picks an older version → selection remembered per domain → searches run on that archive with a banner	Archive browsing never mixes domains
FR-09	Role dashboard	Login → current class, next class with countdown, free periods, later today → cards	Dashboard and typed questions give identical answers
FR-10	Weekly timetable view	Day selection → entries for the user's batch and lab group (and parent division) → list with room/teacher	Lab group A1 does not see A2's labs
FR-11	Natural-language search	English text → parse, build a query plan (operation, filters, lab/lecture, date/week, time window, sort), resolve entities in this timetable, validate, compile with role scope → result envelope	No SQL generated from text; counts, first/last and day-wise questions answered; ambiguous a.m./p.m., date or entity asks for clarification
FR-12	Free-room and professor-availability search	Time/date, optional floor → room inventory minus scheduled rooms → free rooms / free periods	Uncertain entries treated as busy; incomplete inventory reported
FR-13	Conflict detection	Timetable → interval overlap per professor, room, batch, excluding parallel lab groups and electives → conflict list	Real PDF: exactly the 2 genuine clashes reported
FR-14	Personal timetables	Own upload → same pipeline, stored in personal tables → private timetable	Other users (including Admin) get 404 for it

ID	Requirement	Input → processing → output	Acceptance criterion
FR-15	Academic promotion	Admin preview + confirm → transactional update of placements with history → students moved to next year/-batch	All-or-nothing; history preserved; concurrent promotions cannot double-apply
FR-16	Master data management	Admin forms → validated CRUD of departments, years, batches, courses, rooms, aliases → records	Duplicates and invalid references rejected
FR-17	User and account management	Admin → register student/staff, roles, suspend, issue access code → accounts	Suspension revokes sessions immediately
FR-18	Notifications	Activation and job completion → in-app notification (push in store builds) → notification list	Users see only their own notifications
FR-19	Audit log	Sensitive actions → append-only audit events → Admin audit screen	Uploads, activations, corrections, promotions, access denials logged

3.5 Non-Functional Requirements

Table 3.3: Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	Median search response < 300 ms for one user; extraction of a 10-page PDF < 10 s.
NFR-02	Scalability	Stateless API that scales horizontally by adding processes; workers scale independently.
NFR-03	Availability, reliability	Correct answers under concurrent use; idempotent uploads; failed jobs retried up to 3 times; health endpoint for the platform.
NFR-04	Security	Argon2id passwords, JWT + refresh-token rotation and reuse detection, RBAC, IDOR protection, rate limiting, parameterised SQL only, secrets never in the app bundle.
NFR-05	Privacy	Personal timetables visible only to their owner; no free text in logs.
NFR-06	Usability, accessibility	Readable in light and dark mode; large tap targets; plain-English questions; clarifications with tappable choices.
NFR-07	Maintainability	Modular layers (API, services, query, extraction); typed code (Pydantic, TypeScript strict); automated tests; documented decisions.
NFR-08	Portability	One code base for Android, iOS and web; Docker image for any Linux host.
NFR-09	Resource utilisation	One API worker < 300 MB RAM; database < 50 MB for one institute's timetables.
NFR-10	Auditability	Every entry keeps raw text, page, parser version and status; every correction keeps before/after.

3.6 Hardware and Software Requirements

Table 3.4: Hardware and software requirements

Item	Requirement / version used
Development machine	Any laptop with 8 GB RAM, 4 CPU cores (Windows, macOS or Linux)
Server (demo)	Render free web service (0.1 CPU, 512 MB RAM) + free PostgreSQL
Server (recommended)	2 vCPU, 2–4 GB RAM, managed PostgreSQL, HTTPS
Client devices	Android 8+ or iOS 16+ phone; any modern browser
Back-end language	Python 3.13 (3.12+ supported)
Web framework	FastAPI 0.143, Uvicorn, Pydantic v2
Database	PostgreSQL 16, SQLAlchemy 2.1 ORM, Alembic migrations, psycopg 3
Extraction libraries	PyMuPDF, Tesseract OCR (pytesseract), Pillow, openpyxl, python-docx, LibreOffice (legacy formats)
Security libraries	argon2-cffi, PyJWT
Front end	TypeScript (strict), React 19.3, React Native 0.88, Expo SDK 58, Expo Router, TanStack Query, React Hook Form + Zod, Expo SecureStore
Testing	pytest, Jest (jest-expo), Playwright (Chromium), custom load script
Tools	Git and GitHub, VS Code, Docker, Render, Expo Go

3.7 Constraints and Assumptions

Constraints.

- **Financial:** zero budget, so only free and open-source software and free hosting tiers were used. The free tier sleeps after 15 minutes idle and its database expires after 30 days.
- **Technical:** the source of truth is a human-formatted PDF whose layout can change every semester; the parser must tolerate this and report what it cannot read.
- **Operational:** the system must not change the official timetable without explicit Admin activation, and the Admin must never be able to read personal timetables.
- **Time:** one semester alongside regular course work.

Assumptions.

- The timetable repeats weekly within a semester (date-specific entries are supported but rare).

- Users have a college e-mail address issued by the institute.
- Room floors can be derived from the room number (e.g. 508 → floor 5) until the Admin corrects them.
- Interval convention: a class from 9:00 to 10:00 occupies $[9{:}00, 10{:}00)$.

3.8 Use Cases and User Stories

Table 3.5: User stories and expected system responses

ID	User story	Expected system response
US-1	As a **student of SE-A1**, I want to see my next class when I open the app.	Home shows the next class, its room, teacher and a countdown; only A1's labs are included.
US-2	As a student, I want to ask "Do I have classes after lunch on Tuesday?"	Uses the SE lunch row (1:15–2:15) from the timetable and lists classes from 2:15.
US-3	As a student, I ask "Am I free at 2?"	Asks "2 a.m. or 2 p.m.?" and the date, with tappable choices; never guesses.
US-4	As a **professor**, I want my schedule today and my free periods.	Lists classes mapped to my staff record and the gaps within working hours.
US-5	As a professor, I want a free room on floor 6 at 11 am.	Lists rooms on floor 6 with no class overlapping 11:00 that day.
US-6	As the **HOD**, I want to know about clashes before the semester starts.	Conflicts screen lists professor/room/batch overlaps with page references.
US-7	As the **Admin**, I upload the revised PDF and publish it.	Upload → worker extracts → validation summary → review → activate → everyone notified.
US-8	As a student, I upload my coaching-class timetable.	Stored in my personal domain; never mixed with the college timetable; invisible to others.
US-9	As the Admin, at year end I promote SE students to TE.	Preview of moves, confirm once, history kept; next login shows the TE timetable.

Chapter 4: System Architecture and Design

4.1 Overall System Architecture

Figure 4.1 shows the block diagram. The system follows a layered client–server architecture with one separately scaled background worker.

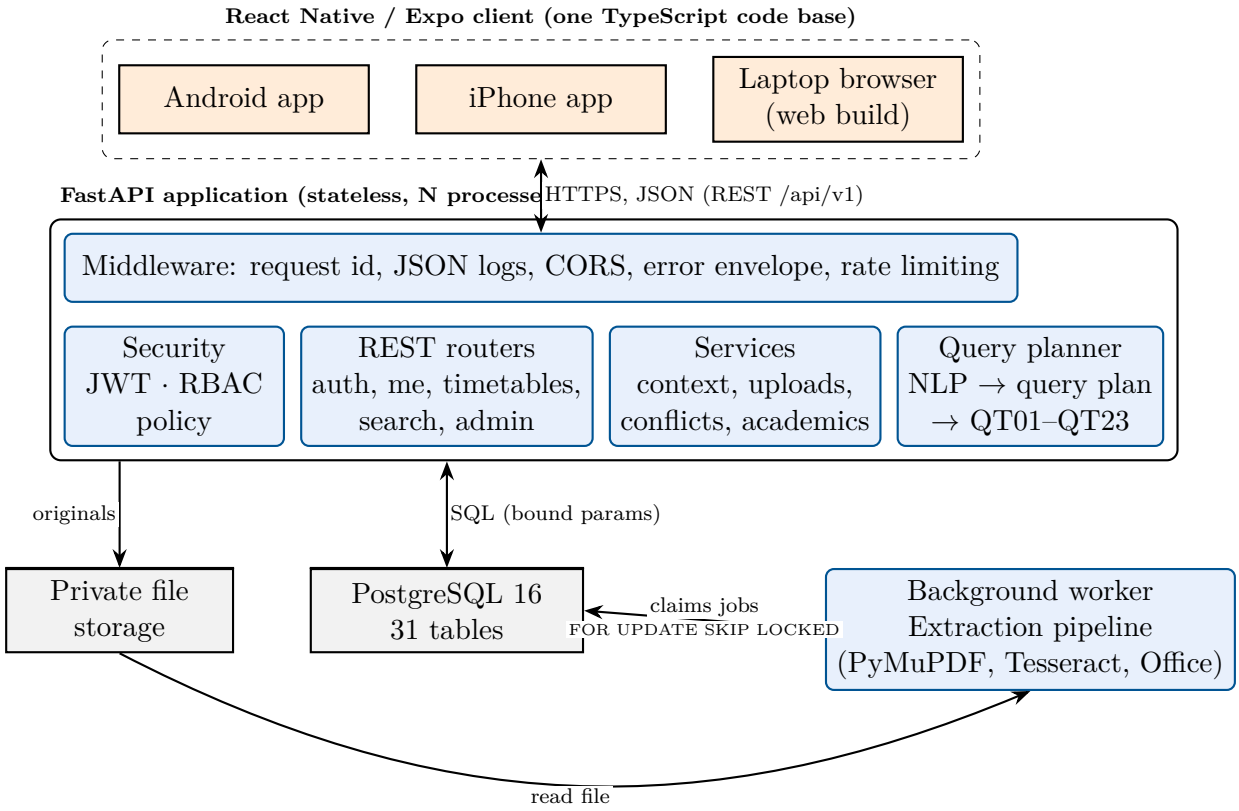


Figure 4.1: Overall system architecture (block diagram)

Responsibilities and interactions. The **client** renders dashboards and forms, caches reads with TanStack Query and keeps the session in the device keystore. It calls the **API** over HTTPS with a bearer token. The API authenticates the user, loads roles from the database, resolves the search context (domain, timetable, time zone, current time), and runs services and the query engine. Uploads are stored privately and a `PROCESSING_JOB` row is inserted in the same transaction. The **worker** claims jobs, extracts and validates the timetable, and writes entries back. **PostgreSQL** is the single source of truth, and its row locks are also used for synchronisation.

4.2 Component and Module Description

Table 4.1: Modules: purpose, inputs, outputs and dependencies

Module (path)	Purpose	Input / output	Depends on
Auth & RBAC `app/security`	Login, tokens, password hashing, role→scope policy, rate limit	Credentials / tokens; request / principal	`USER_ACCOUNT`, `USER_ROLE`, Argon2, PyJWT
API routers `app/api/v1`	HTTP endpoints, request/response validation	JSON, multipart / JSON envelopes	Services, Pydantic
Context resolver `services/context.py`	Chooses domain, timetable, time zone, "now" on the server	User + hints / context object	Preferences, settings
Upload service `services/uploads.py`	Validates file, stores it, creates archive + job	File / timetable id (QUEUED)	File types, storage, jobs
Extraction `app/extraction`	File → grid → sections, times, cells → validated entries	Bytes / `ExtractionResult`	PyMuPDF, Tesseract, openpyxl, python-docx
Worker `app/worker.py`	Claims and runs jobs, heartbeat, retries, status update	Job rows / entries, status, notification	Extraction, DB
Query planner `app/query`	Parse, build a typed query plan, resolve entities, validate, compile into templates (specialised QT01–QT22 or composed QT23)	Text / result envelope	Vocabulary, templates, policy
Conflicts `services/conflicts.py`	Detect professor/room/batch overlaps	Timetable id / conflict list	Entries
Academics `services/academics.py`	Fresher registration, promotion, transitions	Admin commands / history rows	Student history
Admin master data	Departments, years, batches, courses, rooms, aliases, users	Forms / records	DB, audit

Module (path)	Purpose	Input / output	Depends on
Notifications & audit	In-app/push notifications; audit trail	Events / rows	Expo push (optional)
Mobile app `mobile/src`	Screens, navigation, state, API client	User actions / rendered UI	REST API

4.3 Architecture Selection and Justification

We compared three options.

1. **Microservices** (separate auth, extraction and search services with a message broker): independent scaling, but much more deployment and debugging effort for a single-college workload and a team of students, and not possible on a free hosting tier.
2. **Serverless functions**: no server to manage, but cold starts, execution-time limits that do not suit OCR, and vendor lock-in.
3. **Modular monolith + worker** (chosen): one Python service with clearly separated layers (API → services → query/extraction → data). The only CPU-heavy part, extraction, runs in a separate worker process so it never blocks API requests. A PostgreSQL job table replaces a message broker, so job creation is in the *same transaction* as the upload record and there is one less moving part.

The API is **stateless** (REST [16]): any process can serve any request, so the system scales horizontally by adding processes. A **layered** design keeps the rules (security, domain separation) in one place each, which makes them testable.

4.4 Data Flow Diagrams

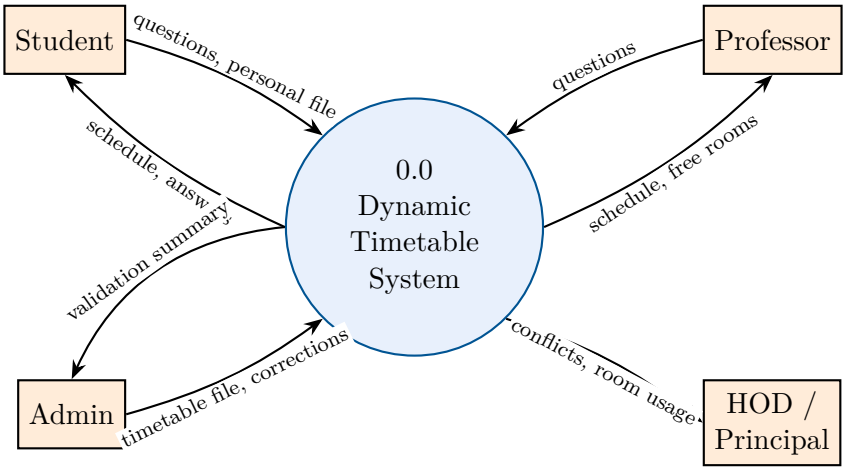


Figure 4.2: DFD level 0 (context diagram)

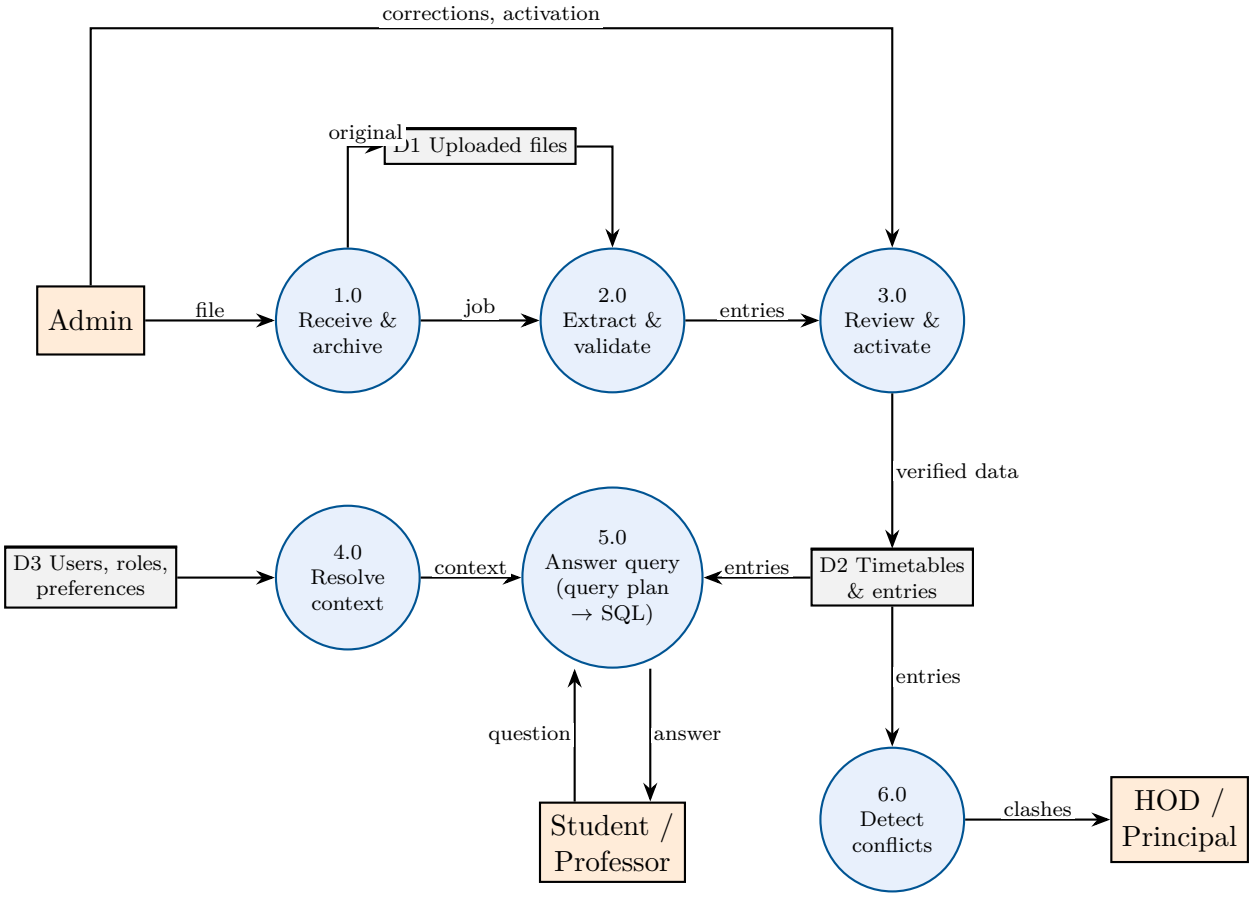


Figure 4.3: DFD level 1

The level-0 DFD shows the system with its four external entities. The level-1 DFD splits it into the upload path (1.0–3.0, used by the Admin) and the query path (4.0–6.0, used by everyone). Both share the timetable store D2. Personal timetables follow the same path but are written to separate personal tables (not drawn).

4.5 UML Diagrams

4.5.1 Use case diagram

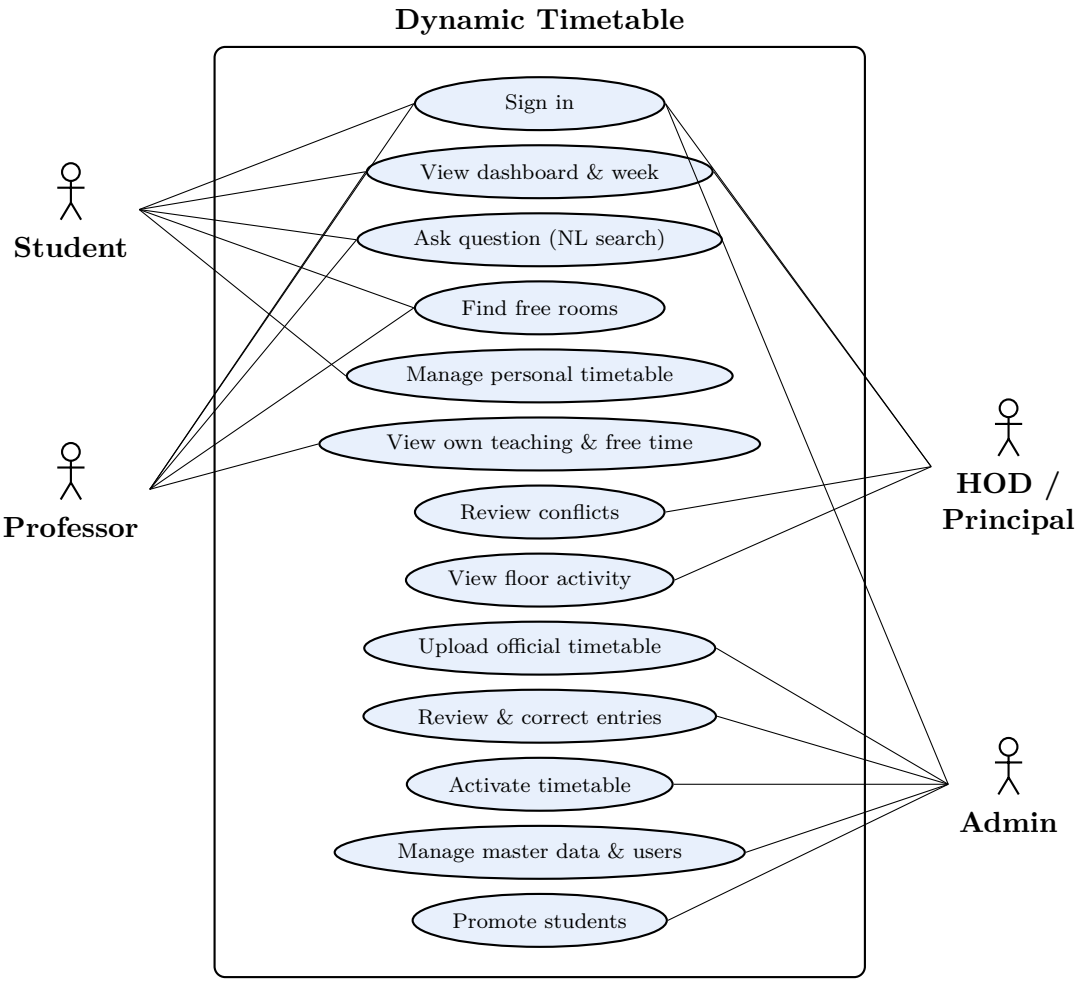


Figure 4.4: Use case diagram

Professors inherit the student-facing use cases. The HOD and Principal are professors with wider scopes, which is why the system uses many-to-many `USER_ROLE` rather than one role per user.

4.5.2 Class diagram

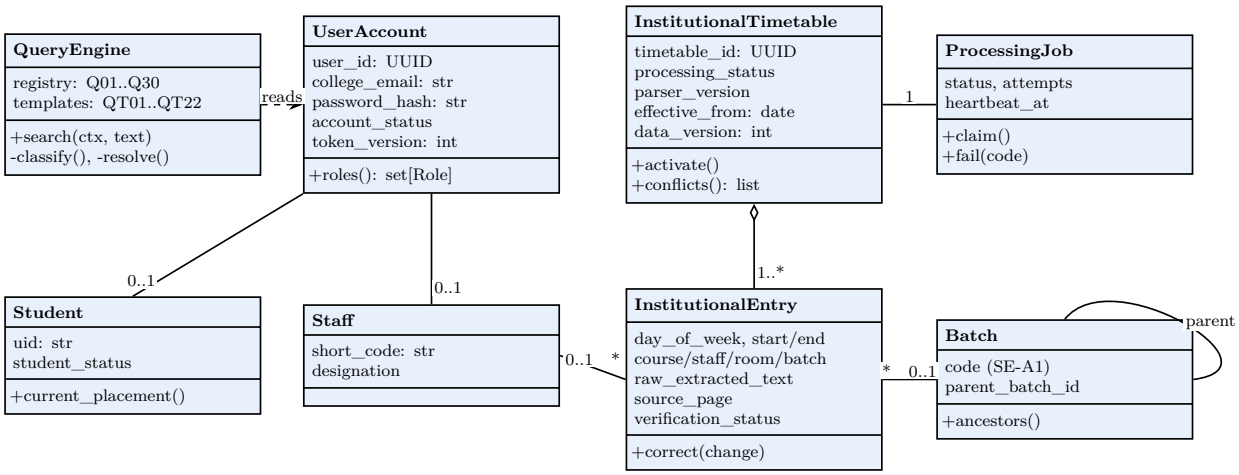


Figure 4.5: Simplified class diagram of the core domain

4.5.3 Sequence diagram: answering a question

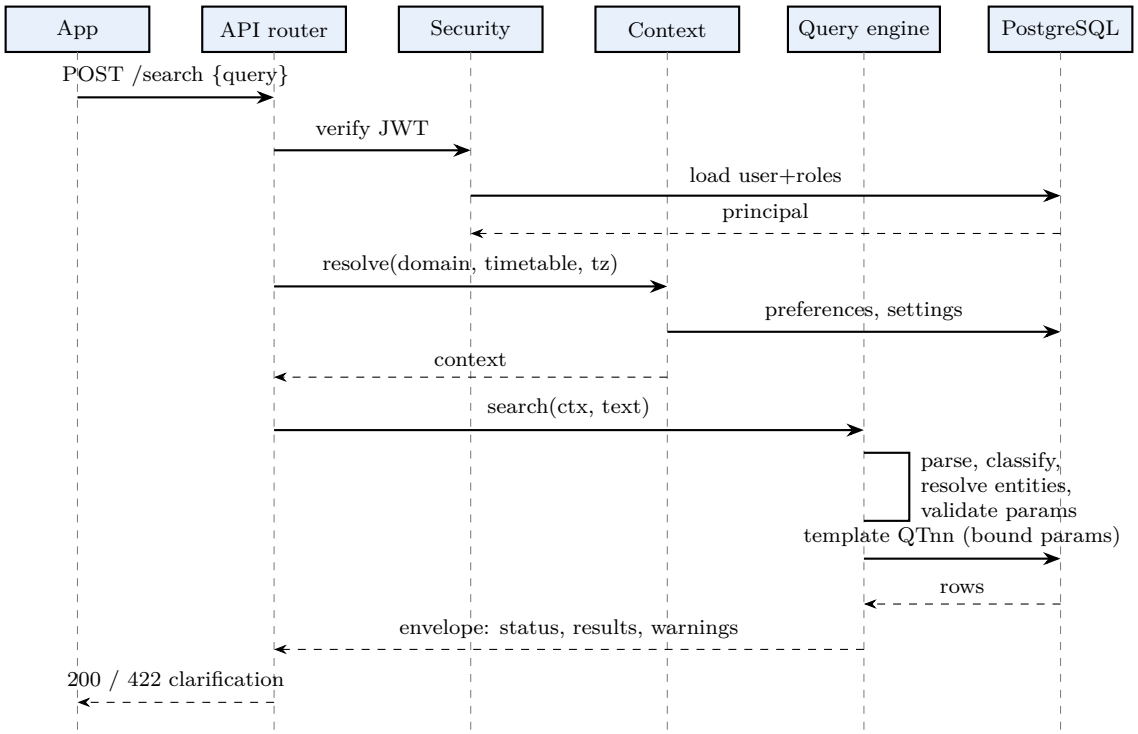


Figure 4.6: Sequence diagram for a natural-language search

Identity, roles and the selected timetable are always resolved on the server, so the client cannot claim another role or timetable. If a parameter is missing (for example a.m./p.m.), the engine

returns `CLARIFICATION_REQUIRED` with choices instead of running a template.

4.5.4 Activity diagram: publishing a new timetable

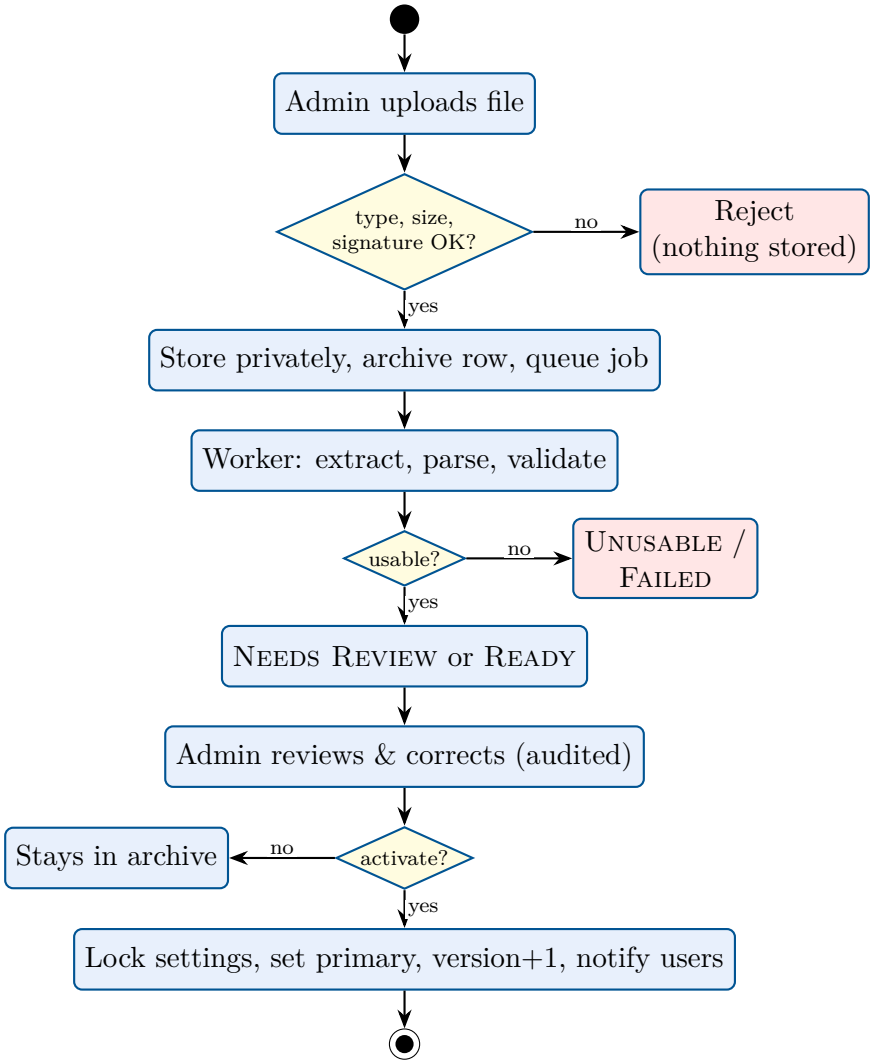


Figure 4.7: Activity diagram: upload, review and activation

4.5.5 State machine diagram: timetable processing status

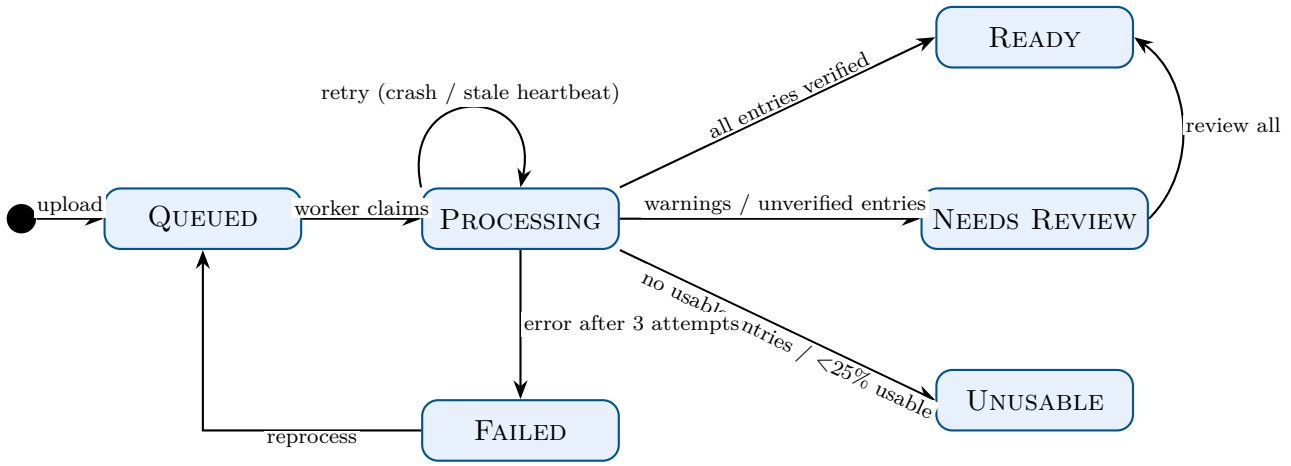


Figure 4.8: State machine of a timetable archive (READY and NEEDS REVIEW may be activated)

4.5.6 Deployment diagram

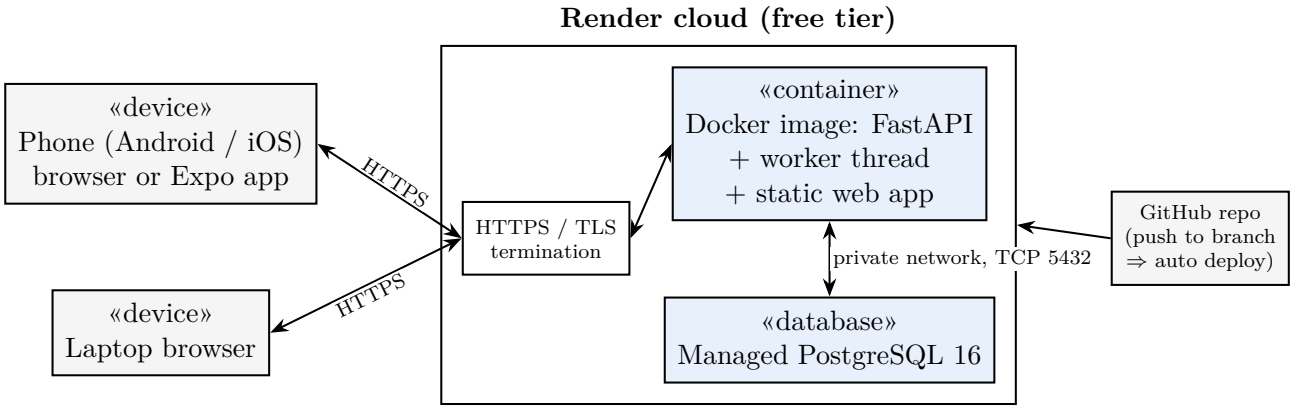


Figure 4.9: Deployment diagram of the free online demo

For the demo, the worker runs as a thread inside the web container, because the free tier allows one service. In production the API and the worker run as separate processes or containers (`docker-compose.yml`).

4.6 Database Design

4.6.1 ER diagram

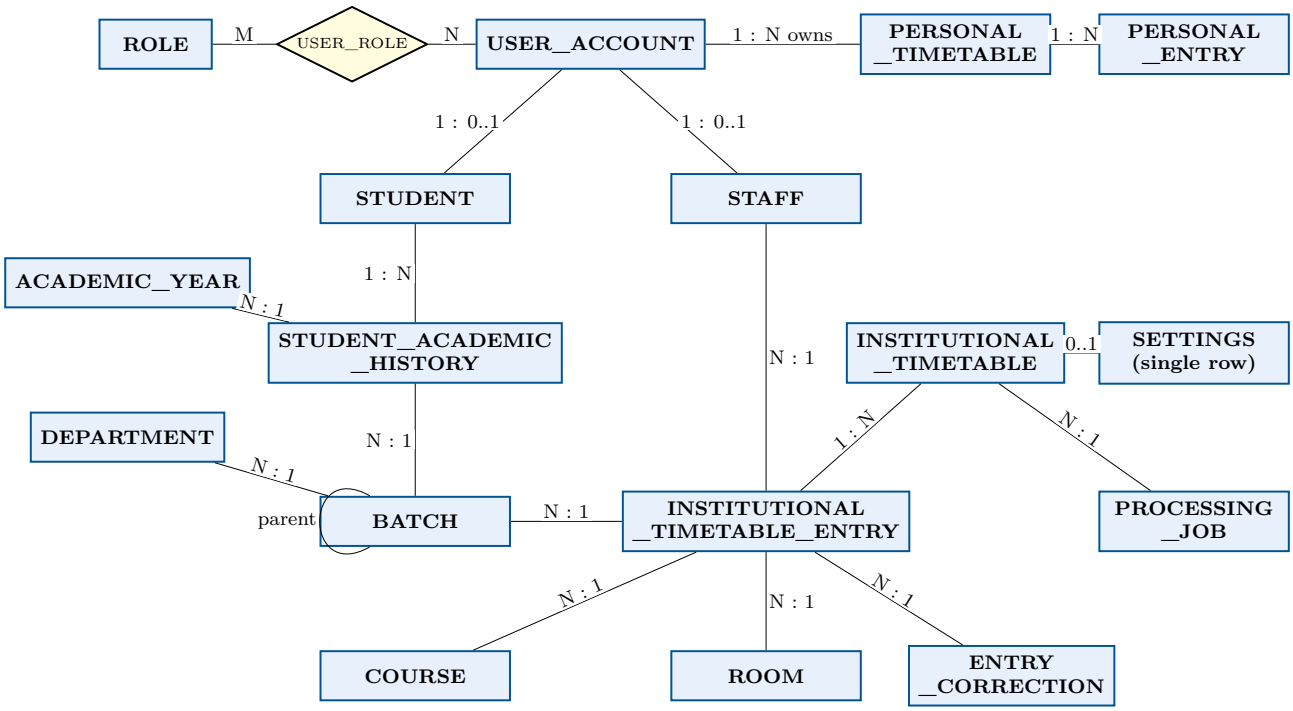


Figure 4.10: ER diagram (main entities; personal sections, notifications, tokens and audit omitted)

4.6.2 Relational schema, keys and constraints

The database has 31 tables, created by three Alembic migrations. Table 4.2 lists the most important tables. Primary keys are UUIDs for user-facing objects (non-guessable, which prevents ID enumeration) and `BIGINT` identities for master data.

Table 4.2: Relational schema of the main tables

Table (PK)	Main attributes	FKs and constraints
`user_account` (user_id)	college_email, display_name, password_hash, account_status, token_version	UNIQUE(college_email)
`user_role` (user_id, role_id)	–	FK user, FK role (M:N bridge)
`student` (student_id)	uid, student_status	FK user (UNIQUE), UNIQUE(uid)

Table (PK)	Main attributes	FKs and constraints
`staff` (staff_id)	short_code, designation	FK user (UNIQUE), FK department
`student_academic_history` (history_id)	year_of_study, semester, status, effective_from/to	FK student, academic_year, department, batch; **partial UNIQUE(student_id) WHERE effective_to IS NULL**
`batch` (batch_id)	code (SE-A1), cohort, division, program_year	FK department, self-FK parent_batch_id; UNIQUE(dept, code)
`institutional_timetable` (timetable_id)	title, file_sha256, file_format, processing_status, parser_version, effective_from, data_version	FK academic_year, department, uploader; partial UNIQUE(uploader, client_request_id)
`institutional_timetable_entry` (entry_id)	entry_kind, day_of_week, class_date, start_time, end_time, raw labels, raw_extracted_text, source_page, confidence_score, verification_status, time_uncertain	FK timetable, section, course, batch, staff, room; CHECK day 1–7; CHECK end>start; CHECK status $\in$ {VERIFIED, UNVERIFIED, INCOMPLETE, REJECTED}; CHECK $0 \leq$ confidence ≤ 1
`institutional_timetable_settings` (settings_id)	default_timetable_id, version, updated_by	**CHECK settings_id = 1** (single row)
`personal_timetable` (timetable_id)	owner_user_id, status, ...	UNIQUE(timetable_id, owner_user_id) for composite FKs
`user_preference` (user_id)	selection_mode, personal_primary_timetable_id, last_active_domain, last institutional/personal timetable	**composite FKs (timetable_id, owner_user_id)**: a user can only point to their own personal timetable
`processing_job` (job_id)	domain, status, attempts, heartbeat_at	CHECK exactly one target matching the domain
`entry_correction` (correction_id)	domain, before/after JSON, reason, corrected_by, corrected_at	FK entry
`audit_event` (event_id)	actor, event_type, target_type/id, request_id, occurred_at	append-only

Normalisation. The schema is in third normal form: course, staff, room and batch details are stored once and referenced by FK; many-to-many relations (users–roles) use bridge tables; academic placement is a separate history table instead of repeating groups in `student`. One deliberate *denormalisation* is that each entry also keeps the raw labels exactly as printed (e.g. `KKD`, `606-4`). This is provenance data rather than derived data: it is needed for review and stays correct when master data changes.

Indexing strategy. Every query template filters by the selected timetable first, so composite B-tree indexes start with `timetable_id`: `(timetable_id, day_of_week, start_time)` for day views, "now" and "next"; `(timetable_id, class_date, start_time)` for dated entries; and `(timetable_id, staff_id)`, `(..., room_id)`, `(..., batch_id)` and `(..., course_id)` for professor, room, batch and course searches. Partial unique indexes enforce business rules (one open placement per student, one active academic year, idempotent uploads). Status indexes support the worker's job claim.

Sample records. Table 4.3 shows real rows extracted from page 1 of the college PDF (SE Division A, Monday).

Table 4.3: Sample entries extracted from the real timetable (page 1, Monday)

Day	Start	End	Course	Staff	Room	Batch	Status
1	09:00	10:00	LLC	–	–	SE-A	VERIFIED
1	10:00	11:00	DBMS	KKD	508	SE-A	VERIFIED
1	11:15	12:15	DSGT	AAG	508	SE-A	VERIFIED
1	12:15	13:15	DS	AVN	508	SE-A	VERIFIED
1	14:15	16:15	COA Lab	JS	606-4	SE-A1	VERIFIED
1	14:15	16:15	COA Lab	AVS	607-B	SE-A2	VERIFIED

4.7 Security and Error Handling Design

Table 4.4: Threats and controls

Threat	Control in the design
Password theft	Argon2id hashing [15]; generic login error; per-IP and per-e-mail rate limit
Stolen refresh token	Rotation on every refresh; reuse of an old token revokes the whole token family
Role escalation	Roles read from `USER_ROLE` on every request; role change bumps `token_version`
IDOR on personal data	Owner check in the query itself; non-owners get 404 (no existence leak) and an `ACCESS_DENIED` audit row
SQL / prompt injection	No SQL built from text; query plans use a closed vocabulary of operations and filters; typed, bound parameters; unknown parameters rejected
Malicious upload	Extension allow-list *and* magic-byte signature check; size/page limits; private storage; parsing isolated in the worker; LibreOffice with isolated profile and timeout
Account takeover via reset	Reset codes never returned in API responses or logs in production (bug P0-1 found and fixed in our audit)
Information leakage	Uniform error envelope with request id; no stack traces or SQL in responses; no tokens or free text in logs
Secrets in the app	Only the public API URL is in the bundle; tokens in iOS Keychain / Android Keystore (SecureStore)

Error handling. Every error returns the same JSON envelope (`code`, `message`, `request_id`). Validation errors return 422 with field details. Query results use application statuses (`OK`, `NO_MATCH`, `CLARIFICATION_REQUIRED`, `UNSUPPORTED_INTENT`, `DATA_UNVERIFIED`), so the app can show a helpful message instead of a generic failure. The worker separates deterministic failures (unreadable file: no retry) from unexpected ones (retried up to three times). Structured JSON logs carry the request id for tracing.

Chapter 5: Algorithm Design and Data Structures

5.1 Identification of Critical Operations

Table 5.1: Critical operations of the system

#	Operation	Why it is critical	Frequency
C1	Time-row resolution	Wrong a.m./p.m. makes every class in the row wrong	Per upload
C2	Cell parsing (stacked labs, rooms, legends)	Determines subject, teacher, room and lab group of every entry	Per upload
C3	Conflict detection	Must find genuine clashes and ignore intended overlaps	Per upload / on demand
C4	Query planning	Every question must become one valid, typed query plan (or a clarification)	Every search
C5	Entity resolution	"Desai", "DBMS", "508" must resolve only within the selected timetable	Every search
C6	Next class / free time / free rooms	Most frequent dashboard operations; must be fast and correct	Every app open
C7	Job claiming	Several workers must never process the same upload twice	Per upload

5.2 Algorithm Design

5.2.1 C1: Time-row resolution by constrained backtracking

Time labels in the real PDF are inconsistent. For example, "12.15 a.m. to 01.15 p.m." appears where noon is meant, and many labels have no a.m./p.m. at all. Each label is turned into a small set of candidate intervals, and the algorithm searches for assignments in which every row is a plausible interval (between 7:00 and 21:00, at most 6 hours long) and no row starts before the previous row ends. A reading is accepted as certain only if it is the *same in every valid assignment*. Otherwise the row is marked uncertain and sent to review. This follows the project rule that the system never guesses silently.

Algorithm 1 Resolve a column of time labels

Input: labels $L_1..L_r$ (top to bottom); each L_i gives start/end hour, minute and optional marker
Output: for each row: (start, end, certain?) or a review flag

for $i \leftarrow 1$ **to** r **do**
  $O_i \leftarrow \{(s,e) : s \in \textsc{Cand}(L_i.start), e \in \textsc{Cand}(L_i.end), \textsc{Plausible}(s,e)\}$ ▷ literal markers honoured
  if $O_i = \emptyset$ **then** $O_i \leftarrow$ readings ignoring markers, flagged *non-literal*
  end if
end for
$S \leftarrow \emptyset$
procedure $\textsc{Walk}(i, prevEnd, A)$
  if $|S| > 64$ **then return**
  end if ▷ bound the search
  if $i > r$ **then** $S \leftarrow S \cup \{A\}$; **return**
  end if
  for all $(s,e) \in O_i$ **with** $s \geq prevEnd$ **do**
    $\textsc{Walk}(i+1, e, A \cdot (s,e))$
  end for
end procedure
$\textsc{Walk}(1, 0, \langle\rangle)$
for $i \leftarrow 1$ **to** r **do**
  $R_i \leftarrow \{A_i : A \in S\}$
  if $|R_i| = 1$ **and** reading is literal **then** mark row i CERTAIN
  else if $|R_i| = 1$ **then** keep reading, set `time_uncertain`, add `TIME_LABEL_INCONSISTENT`
  else leave time empty, add `TIME_AMBIGUOUS_AMPM` (entries become INCOMPLETE)
  end if
end for

A special rule handles the college's "12.15 a.m. to 01.15 p.m." row. It is read as midday only when the row before it ends exactly at 12:15 and the row after it starts exactly at 1:15. It is therefore proven by its neighbours, not assumed.

5.2.2 C2: Cell parsing

Algorithm 2 Parse one grid cell into entries

Input: cell text c, row time (s, e), day d, section scope, page legend Λ
Output: list of entries with raw text and status

1: *parts* $\leftarrow$ split c into lines; group lines that belong to one lab sub-group (e.g. `A1 COA Lab JS 606-4`)
2: **if** c is a break / lunch / activity **then return** one BREAK/ACTIVITY entry
3: **end if**
4: **for all** $p \in$ *parts* **do**
5: tokens $\leftarrow$ split p on "/", "," and whitespace
6: $g \leftarrow$ sub-group token matching `[A-D][1-4]` or `Batch`n (if any)
7: $\rho \leftarrow$ last token matching a room pattern (`508`, `606-4`, `509 old`)
8: $\phi \leftarrow$ tokens found in Λ as faculty codes; $\kappa \leftarrow$ remaining subject tokens
9: expand κ, ϕ with Λ only if the code exists *on this page*; else keep code + `INFO`
10: inline range "(2.15–4.15 pm)" overrides (s, e) if consistent; else flag a conflict
11: emit entry$(d, s, e, \kappa, \phi, \rho, \text{scope} + g)$ with raw text, page and cell region
12: **end for**
13: VALIDATE each entry $\rightarrow$ VERIFIED/UNVERIFIED/INCOMPLETE/REJECTED

5.2.3 C3: Conflict detection (group, sort and sweep)

Algorithm 3 Detect professor, room and batch clashes

Input: entries E of one timetable (classes with day and times, not rejected)
Output: list of conflicts with confidence CONFIRMED/POSSIBLE

```
G ← empty hash map
for all x ∈ E do
    for all resource (k, key) ∈ {professor, room, batch} of x do
        append x to G[(k, key, x.day)]          ▷ id if mapped, else normalised label
    end for
end for
for all group g ∈ G do
    sort g by start time
    for i ← 1 to |g| do
        for j ← i + 1 to |g| do
            if g_j.start ≥ g_i.end then break          ▷ sorted: no later entry can overlap
            end if
            if g_i.start < g_j.end ∧ g_j.start < g_i.end and not INTENDED(g_i, g_j) then
                report (k, g_i, g_j); CONFIRMED if both verified, mapped and certain
            end if
        end for
    end for
end for
function INTENDED(a, b): same source cell, same combined session on several pages, parallel
elective options, or a sub-group's own written period
end function
```

On the real PDF a naive pairwise check reported 542 overlaps. Adding the intended-overlap rules in two refinement steps (first 25, then 2) left exactly the **2 genuine clashes**, which we confirmed by hand against the PDF (Section 10.3).

5.2.4 C4 and C5: Query planning and interpretation

The query engine does not match a whole question to one of a fixed list of question types. A *planner* recognises independent pieces of the question and combines them into a typed **query plan**: an *operation* (LIST, COUNT, EXISTS, TOTAL_HOURS, FIRST, LAST, NEXT, PER_DAY), *filters* (me, batch, professor, room, course; lab or lecture), *when* (a date, a weekday, this/next week or the whole weekly timetable), a *time window* (at, after, before, between, after lunch) and, for day-wise questions, a *metric and sort* (most/least classes or hours, earliest/latest start or finish). For example, "Which day do I finish earliest?" becomes `{operation: PER_DAY, metric: LAST_END, sort: EARLIEST}` and "How many labs do I have this week?" becomes

`{operation: COUNT, kind: LAB, when: {week: this}}`. Questions that need dedicated logic (free rooms, a professor's free time or scheduled location, room status, archives ...) keep their specialised operations (templates QT01–QT22); all other plans are compiled by the composed executor (template QT23) from the same building blocks.

Algorithm 4 Build a query plan from the parsed question

Input: parsed question p, specialised operation S recognised by the rule cascade (or $\perp$)
Output: plan P

1: **if** S is pinned (archives, primary, cross-domain, time-until) **then return** $P(S)$
2: **end if**
3: $k \leftarrow$ LAB / LECTURE from words ("lab", "practical" / "lecture", "theory"); $\perp$ otherwise
4: $w \leftarrow$ window from times and words (between, before, after, after lunch, at)
5: $o \leftarrow$ TOTAL_HOURS ("total hours") $\vee$ COUNT ("how many") $\vee$ PER_DAY ("which day", "busiest") $\vee$ NEXT / FIRST / LAST $\vee$ EXISTS ("any lab")
6: **if** $o =$ PER_DAY **then** choose metric (finish $\rightarrow$ LAST_END, start $\rightarrow$ FIRST_START, else classes/hours) and sort (most/least, earliest/latest)
7: **end if**
8: **if** $o =$ COUNT and S returns rooms/professors/courses **then return** P(COUNT over S)
9: **end if**
10: **if** $o = \perp$ and $k = \perp$ and $w \neq$ BEFORE **then return** $P(S)$ ▷ specialised operation unchanged
11: **end if**
12: **return** composed $P(o$ or LIST$, k, w,$ filters, when, sort$)$

Algorithm 5 Answer a natural-language question

Input: text q, authenticated principal u, optional domain hint
Output: response envelope

$p \leftarrow$ Parse(q): normalise; extract dates, weekdays, times (with ambiguity), "now", "after lunch", anchored mentions ("prof X", "room 508", "5th floor")
if p asks to compare personal and college data **then return** `UNSUPPORTED_INTENT` (Q30)
end if
$ctx \leftarrow$ ResolveContext(u, hint) ▷ domain, timetable, time zone, now; server side
$V \leftarrow$ vocabulary of labels of *this* timetable only
$M \leftarrow$ Scan(p, V): longest-first n-gram match ($n = 4..1$) of unanchored mentions
$S \leftarrow$ Classify(p, M): ordered rules recognise a specialised operation (or $\perp$)
$I \leftarrow$ BuildPlan(p, S) (Algorithm 4)
if $I = \perp$ **then return** `UNSUPPORTED_INTENT` with example questions
end if
check I allowed for role and domain; check student/staff mapping exists
for all required parameter π of I **do**
    if π missing or ambiguous (a.m./p.m., date, several entities) **then return** `CLARIFICATION_REQUIRED` with choices
    end if
end for
rows $\leftarrow$ compile I: specialised template, or composed QT23 (days $\times$ filters $\times$ window, then count / sum / first / last / per-day), always with bound parameters and role scope
return `OK` / `NO_MATCH` / `DATA_UNVERIFIED` with warnings

Entity resolution (C5) works in this order: (1) exact match of normalised label, id or alias; (2) a *unique* whole-word match ("desai" → Prof. Kiran K. Desai); (3) fuzzy suggestions (RapidFuzz similarity) that are *always shown as choices*, never auto-selected. Teachers are found the way students say them: faculty code ("AVN"), full name, first name or surname alone ("Anant", "Nimkar's schedule"), or with "sir"/"ma'am" after it. A surname shared by several teachers ("Sawant") asks which one; entries that share a faculty code are merged into one person, because the PDF legend sometimes spells the same name differently. Subjects are found by part of their name ("database", "computer organization") with a one-letter typo tolerance for long words.

5.2.5 C6: Next class and free rooms

Algorithm 6 Next class and free rooms

function NextClass(scope, now, $H = 7$)
  for $k \leftarrow 0$ **to** H **do** ▷ today, then following days (one weekly cycle)
    $D \leftarrow now.date + k$; $t_0 \leftarrow (k = 0)$? $now.time$: 00:00
    $x \leftarrow$ first entry in scope with ($class_date = D \vee (class_date = \perp \wedge dow =$ isodow$(D))$), start $> t_0$, time certain, ordered by start ▷ index ($timetable, dow, start$)
    if $x \neq \perp$ **then return** x
    end if
  end for
  return `NO_MATCH`
end function
function FreeRooms(D, t, floor)
  $B \leftarrow \{x.room : x$ on $D,\ x.start \leq t < x.end\} \cup \{$rooms of uncertain entries$\}$ ▷ uncertain = busy
  return $\{r \in$ Rooms(floor) $: r \notin B\}$; warn if some room labels are unmapped
end function

5.2.6 C7: Job claiming

Listing 1: Job claim executed by every worker (simplified)

```
SELECT * FROM processing_job
 WHERE status = 'QUEUED'
    OR (status = 'RUNNING' AND heartbeat_at < now() - interval '10 minutes')
 ORDER BY created_at          -- FIFO
 LIMIT 1
 FOR UPDATE SKIP LOCKED;      -- a row locked by another worker is skipped, not
   waited for
```

5.3 Flowcharts

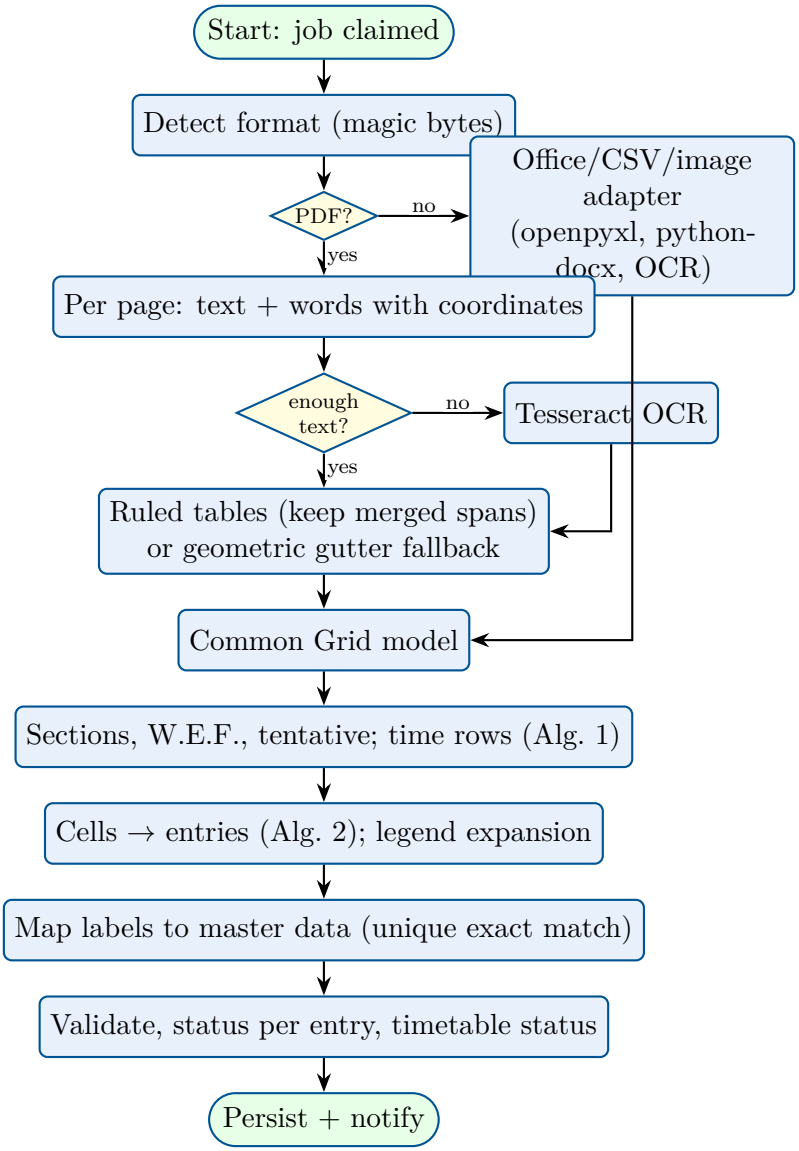


Figure 5.1: Flowchart of the extraction pipeline

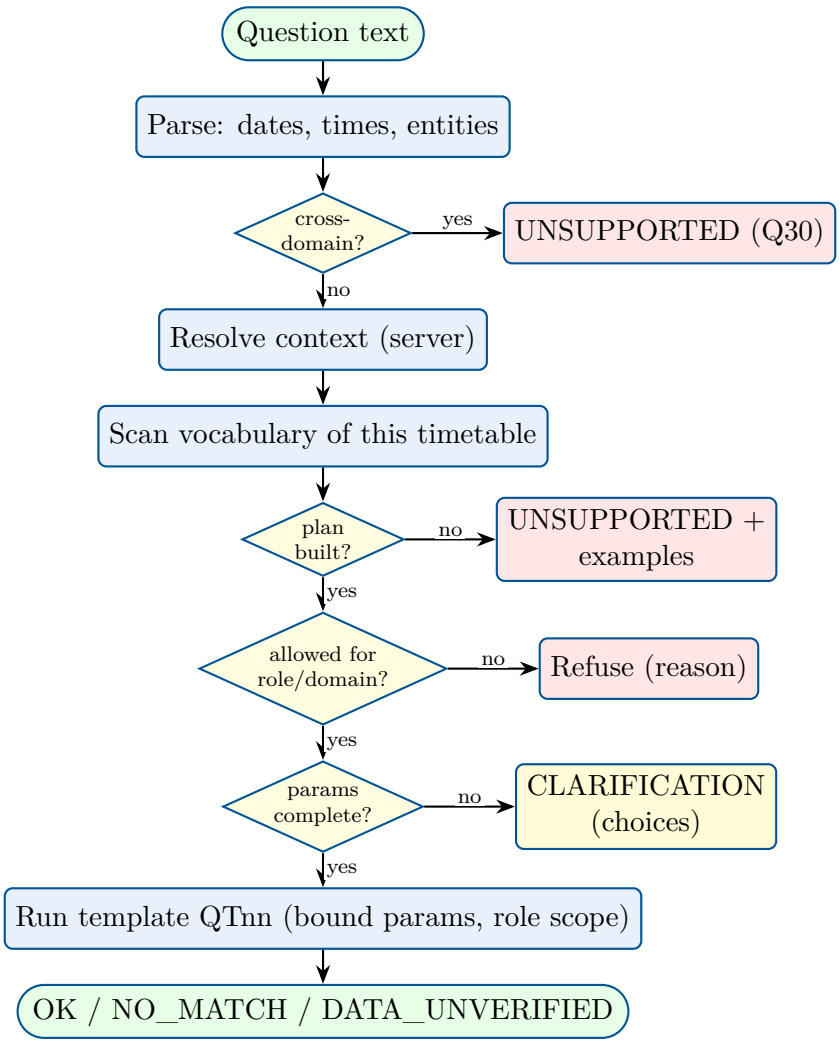


Figure 5.2: Flowchart of the query engine

5.4 Data Structure Selection

Table 5.2: Data structures used and why

Data structure	Where	Reason
Hash map (dict)	Conflict groups keyed by (resource, key, day); page legend; vocabulary n-gram index	$O(1)$ average lookup and grouping, replacing $O(n^2)$ all-pairs comparison
Hash set	Seen conflict pairs; used tokens; role sets	De-duplication in $O(1)$
Sorted array + sweep	Entries per group sorted by start time	Early `break` when no later interval can overlap
2-D grid (list of rows of cells with span)	`Grid` model shared by all adapters	Natural model of a timetable; merged cells keep row/column span
Tree (parent pointer)	Batch hierarchy SE → SE-A → SE-A1	A lab group inherits division classes by walking to the root (O(depth), depth ≤ 3)
Queue (FIFO)	`processing_job` ordered by `created_at`	Fair processing order; persistent across crashes
B-tree index	PostgreSQL composite indexes	$O(\log n)$ range scans on (timetable, day, start)
Search tree of assignments	Time-row backtracking (Alg. 1)	Explores consistent readings; pruned by ordering constraint
Immutable registry (tuple/dict)	Operations Q01–Q31, templates QT01–QT23, plan vocabulary	Closed set; unknown operations or plan values cannot be executed

5.5 Algorithm Selection and Justification

Table 5.3: Alternatives considered for the key algorithms

Problem	Alternative	Chosen	Justification
Clash detection	All pairs, $O(n^2)$	Group by resource (hash) + sort + sweep	Same result, $O(n \log n + m)$; simpler to explain intended-overlap rules per group
Clash detection	Interval tree	Sorted sweep	Groups are small (tens of entries); a tree adds code without measurable gain
Time labels	Trust printed a.m./p.m.	Constrained backtracking with uniqueness	Printed markers are wrong in the real PDF; uniqueness gives provable certainty
Question understanding	LLM text-to-SQL; trained intent classifier; one fixed rule per question	Compositional query plan (independent rules per piece) + vocabulary scan + compiled templates	Deterministic, explainable, injection-safe, no training data or API cost [8, 12]
Entity matching	Fuzzy auto-select	Exact → unique word → fuzzy *suggestions*	Avoids silently picking the wrong professor
Job distribution	Redis/RabbitMQ	PostgreSQL `SKIP LOCKED`	One less service; job creation is transactional with the upload
Table extraction	Deep-learning table detector	PyMuPDF ruled tables + geometric fallback	Text PDFs are exact; no GPU; explainable failures

5.6 Time Complexity Analysis

Let n be the number of entries in a timetable, r the rows of a grid section, w the words in a question, $|V|$ the vocabulary size, m the number of overlapping pairs reported, and R the number of rooms.

Table 5.4: Time complexity of critical operations

Operation	Complexity	Remarks
Time-row resolution	worst $O(4^r)$, bounded by 64 solutions; typical $O(r)$	$r \leq 12$ rows, ≤ 4 readings each; the ordering constraint prunes almost all branches
Cell parsing (whole PDF)	$O(C \cdot \ell)$	C cells, ℓ tokens per cell; legend lookup $O(1)$
Conflict detection	$O(n \log n + m)$ average	Hash grouping $O(n)$, sort per group, sweep with early break. Worst case (all overlapping in one group) $O(n^2)$
Division vs sub-group check	$O(d^2)$ per day	d entries with batch label on a day; acceptable for $d \approx 100$, see Section 5.8
Parse + classify question	$O(w)$	Fixed number of regular expressions
Build query plan	$O(w)$	One pass of a fixed set of piece rules (operation, kind, window, sort)
Execute composed plan	$O(D \log n + k \log k)$	$D \leq 7$ day range scans on the index; k matching entries sorted for slots, first/last and per-day sums
Vocabulary scan	$O(4w)$ lookups	n-gram sizes 4..1, hash index built in $O(\|V\|)$
Next class	$O(H \log n)$	At most H=7 index range scans
Free rooms	$O(\log n + k + R)$	k entries at that time, set difference over R rooms
Job claim	$O(\log J)$	Index on (status, created_at)

5.7 Space Complexity Analysis

Extraction holds one document's grid in memory: $O(C)$ cells plus $O(n)$ entries, a few megabytes for a 10-page PDF. OCR is the largest consumer (one page image at native resolution at a time, so memory is bounded by one page). Conflict detection uses $O(n)$ for the hash groups and $O(m)$ for results. The backtracking stores at most 64 assignments of length r, so $O(64r)$. The query vocabulary is $O(|V|)$ per request (about 200 labels for the real timetable). The database stores one row per entry, so 570 rows (453 classes plus breaks and activities) for the real PDF. The whole development database with this timetable, users and history measured **12 MB**.

5.8 Optimisation Opportunities

- **Done – N+1 queries removed:** loading related course/staff/room/batch rows eagerly cut the number of SQL statements per search from about 50 to about 25 (Section 8.9).
- **Done – batched vocabulary loading:** one query per entity type instead of one per label.

- **Possible – division-pair sweep:** the $O(d^2)$ loop can use the same sort-and-break sweep as the main groups.
- **Possible – vocabulary cache:** cache the vocabulary per (timetable, data_version); it changes only on correction or activation.
- **Possible – parallel page extraction:** pages are independent, so OCR of scanned PDFs could use a process pool.

Chapter 6: Subject-Wise Engineering Concepts and APNI Leap Alignment

This chapter shows how concepts from Semesters I–IV were applied in the project. Every claim points to code in the repository or to a test, so it can be checked.

6.1 Software Engineering

- **Process model:** incremental. Each phase (requirements → back-end foundation → extraction → query engine → app → testing → deployment) produced a working, tested increment and a commit (Section 7.8) [23].
- **Requirement traceability:** every functional requirement (FR-01 . . . FR-19) maps to modules (Table 4.1) and test cases (Table 9.2). Conflicts between the source specifications were recorded with their resolution (`docs/01-requirements-and-decisions.md`, C-01 . . . C-10, D-01 . . . D-20).
- **Modularity and separation of concerns:** routers → services → query/extraction → models. Security policy lives in one file (`security/policy.py`), and domain separation in one adapter (`services/domains.py`).
- **State management (client):** server state in TanStack Query (cache, invalidation on activation), authentication state in a React context, form state in React Hook Form with Zod schemas.
- **Exception handling:** typed application errors mapped to one JSON error envelope; the worker separates retryable from deterministic failures.
- **Version control:** Git with a feature branch and descriptive commit messages (Section 7.8).
- **Quality:** static typing (Pydantic, TypeScript `strict`), 174 + 20 automated tests, end-to-end tests, a security audit with prioritised findings (P0–P2).

6.2 Computer Networks

- **Architecture:** client–server over TCP/IP. Many clients (phones and browsers) talk to one logical server over HTTPS (HTTP/1.1 over TLS over TCP, port 443). The database is reached over TCP port 5432 on a private network.
- **Protocol and message format:** REST resources under `/api/v1`; JSON request and response bodies; `multipart/form-data` for uploads; standard status codes (200, 401, 403,

404, 413, 422, 429). Search answers always use one envelope (`status`, `intent`, `results`, `warnings`, `clarification`).

- **Connection management:** stateless requests with bearer tokens; keep-alive handled by the HTTP client; a database connection pool per API process.
- **Error recovery:** client timeouts; automatic retry only for idempotent reads; *single-flight* token refresh, where many requests that receive 401 at the same time share one refresh call (tested in Jest); offline detection with NetInfo and a cached last-known institutional timetable.
- **Real bug found:** the web build checked connectivity with an HTTP `HEAD /` request, which the server answered with 405, so the app believed it was offline and stayed on "Loading…". The server now accepts `HEAD` for the web routes (Section 9.9).
- **Concurrency:** several Uvicorn worker processes each serve concurrent requests, so many clients are served at once (Section 8.4).

6.3 Database Management Systems

- **ER modelling and normalisation** to 3NF (Section 4.6), with a justified provenance denormalisation (raw labels).
- **Keys and constraints:** UUID and identity PKs; FKs including *composite* FKs that make a user's preference point only to their own personal timetable; CHECK constraints (day 1–7, end > start, status enums, single-row settings); *partial unique* indexes (one open academic placement per student; one active academic year; idempotent uploads per client request id).
- **Indexing and query optimisation:** composite B-tree indexes led by `timetable_id`; eager loading removed N+1 queries (about 50 → about 25 statements per search).
- **Transactions and concurrency control:** activation locks the settings row (`SELECT ... FOR UPDATE`); promotion locks student rows in a stable order to avoid deadlocks; job claiming uses `FOR UPDATE SKIP LOCKED`; each request runs in one transaction, so it sees one consistent snapshot.
- **Schema migration:** Alembic versioned migrations (0001 schema, 0002 reference data, 0003 time-uncertainty flag).
- **Security:** parameterised queries only, so no SQL is ever built from user text.

6.4 Operating Systems

- **Processes:** the API runs as several OS processes (Uvicorn workers); extraction runs in a separate worker process, so CPU-heavy OCR cannot starve request handling. On the free tier the worker runs as a background *thread* in the same process instead.
- **Producer–consumer:** the API (producer) inserts jobs and workers (consumers) remove them from a shared, persistent FIFO queue: the `processing_job` table.
- **Synchronisation:** database row locks act as the mutual-exclusion mechanism. `SKIP LOCKED` gives non-blocking acquisition, which is like a *try-lock*, so idle workers never wait on a busy row.
- **Deadlock avoidance:** resources (student rows) are always locked in the same order, which breaks the circular-wait condition.
- **Fault tolerance:** heartbeat timestamps detect crashed workers, a job whose heartbeat is older than 10 minutes is re-claimed, and the maximum attempt count prevents infinite retry loops.
- **Scheduling:** jobs are served first-come-first-served (`ORDER BY created_at`). The OS scheduler time-slices request handling across processes. Section 8.5 discusses the measurements. We did not need a custom Round-Robin scheduler, because jobs are rare and short (under 2 s).
- **Resource limits:** upload size (15 MB) and page count (30), and a timeout on the LibreOffice conversion subprocess.

6.5 Data Structures

Hash maps for grouping and legend/vocabulary lookup, sets for de-duplication, sorted arrays with a sweep for interval overlap, a 2-D grid with spans for the timetable, a parent-pointer tree for the batch hierarchy, a FIFO queue for jobs and B-tree indexes in the database (Table 5.2).

6.6 Design and Analysis of Algorithms

- **Backtracking with pruning** for time-row resolution, with a uniqueness test for certainty (Algorithm 1).
- **Divide by key, sort and sweep** for interval-overlap detection, $O(n \log n + m)$ instead of $O(n^2)$ (Algorithm 3) [21].
- **Greedy longest-match** n-gram scanning for entity mentions.

- **Compositional semantic parsing**: independent rules for operation, filters, time window and sort combine into a query plan, which is then compiled into queries (a small query algebra: selection by day and filters, then aggregation by count, sum, min/max or group-by-day; Algorithm 4).
- **Interval algebra** for overlap and free-time computation [13].
- **Correctness argument (conflicts):** after sorting by start time, if $g_j.start \geq g_i.end$ then every later g_k $(k > j)$ also has $g_k.start \geq g_i.end$, so it cannot overlap g_i. The early break therefore never misses a conflict.

6.7 Cloud Computing

- **Service model:** Platform-as-a-Service (Render) running our **Docker** container, plus a managed PostgreSQL database (Database-as-a-Service).
- **Infrastructure as code:** `render.yaml` blueprint declares the web service, the database and environment variables. A push to the branch triggers build and deploy (continuous deployment).
- **Configuration and secrets:** 12-factor style environment variables; the secret key is generated by the platform and the demo password is entered at deploy time, so nothing secret is in Git.
- **Availability and scaling:** a health-check endpoint; the stateless API scales horizontally; the free tier sleeps after 15 minutes idle (a cost/availability trade-off).

6.8 Mapping of Subjects to Project Implementation

Table 6.1: Mapping of Semester I–IV subjects to the project

Subject	Concept applied	Project module	Evidence
DBMS	Normalisation, PK/FK, CHECK, partial unique indexes	Schema (`models.py`, migrations)	ER diagram Fig. 4.10; Table 4.2
DBMS	Indexing, N+1 elimination	Query templates	50 → 25 statements per search; latency results (Ch. 8)

Subject	Concept applied	Project module	Evidence
DBMS	Transactions, row locking	Activation, promotion	Concurrent activation/promotion tests (one wins, no corruption)
Operating Systems	Producer–consumer, mutual exclusion (try-lock)	Worker, job queue	`SKIP LOCKED` claim; worker retry test
Operating Systems	Processes vs threads, deadlock avoidance	API workers, promotion	Load test with 4 processes; ordered locking
Computer Networks	Client–server, HTTP/REST, JSON, TLS	API, app client	OpenAPI spec (`docs/openapi.json`); e2e tests
Computer Networks	Error recovery, single-flight refresh	`api/client.ts`	Jest test with concurrent 401s
Data Structures	Hash maps, sets, trees, queues, grids	Extraction, conflicts, vocabulary	Code; Table 5.2
Analysis of Algorithms	Backtracking, sort + sweep, complexity	Time parser, conflicts	Alg. 1, 3; Table 5.4
Software Engineering	SRS, traceability, testing, Git	Whole project	FR table, test-case table, commit history
Discrete Mathematics	Sets, relations, intervals, predicate logic	Clash rule, role scopes	$a.s < b.e \wedge b.s < a.e$; RBAC matrix
Programming (Python, JS)	OOP, typing, async UI	Back end, app	10.5k lines Python, 5.5k lines TypeScript
Cloud Computing	PaaS, Docker, IaC, CD	Deployment	`render.yaml`, `deploy/render.Dockerfile`; live demo
Information Security (applied)	Hashing, tokens, RBAC, OWASP	`app/security`	Auth tests; P0 fix verified with 9 checks

Chapter 7: Implementation and Module-Wise Coding

7.1 Development Environment

- **Languages:** Python 3.13 (back end, about 10,500 lines in `backend/app`), TypeScript in strict mode (app, about 5,500 lines in `mobile/src`), SQL (migrations, templates).
- **Frameworks and libraries:** FastAPI, SQLAlchemy 2.1, Alembic, Pydantic v2, PyMuPDF, pytesseract, openpyxl, python-docx, RapidFuzz, argon2-cffi, PyJWT; Expo SDK 58, React Native 0.88, Expo Router, TanStack Query, React Hook Form, Zod.
- **Tools:** VS Code, Git/GitHub, PostgreSQL 16, Docker, Expo Go, Chrome DevTools, pytest, Jest, Playwright, Render.
- **Operating systems:** Windows and Linux laptops for development; Android and iPhone for device testing.

7.2 Overall Implementation Strategy

The design was translated into software *risk first*, from the inside out:

1. **Foundation:** schema, migrations, authentication, RBAC and admin master data, so every later feature was secure from the start.
2. **Extraction engine:** the riskiest part, built and tuned on the real college PDF early.
3. **Query engine:** first specialised operations and templates, tested one by one (Q01–Q30); then generalised into a compositional query planner (Q31, template QT23) so that questions are not limited to a fixed list.
4. **Mobile app** on top of a stable, documented API (OpenAPI).
5. **Hardening:** concurrency tests, a security audit, the real-PDF rules from the college, laptop/web support, the SDK upgrade and free cloud deployment.

Each step ended with all tests passing and a commit (Table 7.1).

7.3 Module-Wise Implementation

M1. Authentication and RBAC (`app/security`, `services/auth.py`). *Input/Output:* credentials → access token (15 min) and refresh token (30 days, stored only as a hash). *Logic:* Argon2id verification; on refresh, the old token is revoked and a new one is issued in the same family, and reuse of a revoked token revokes the family. `load_principal` reads roles, staff

profile and academic placement from the database on every request. `policy.py` maps roles to data scopes (batch, department, college). *Tests:* 15 tests in `test_auth.py` (rotation, reuse detection, suspension, role change, rate limit).

M2. Upload and archive (`services/uploads.py, storage.py`). *Input:* multipart file plus metadata. *Logic:* extension allow-list and magic-byte check (`extraction/filetypes.py`), size and page limits, atomic private write (mode 0600), SHA-256 for duplicate detection, archive row and job row in one transaction, idempotency via `client_request_id`. *Output:* timetable id in state QUEUED. *Tests:* `test_uploads_and_domains.py` (13).

M3. Extraction (`app/extraction`, 11 files). `pdf.py` (PyMuPDF tables and words), `geometric.py` (gutter-based columns), `ocr.py` (Tesseract), `office.py` (XLSX/CSV/-DOCX, LibreOffice for legacy formats) → `model.py` (Grid) → `sections.py`, `timeparse.py`, `gridparse.py`, `cellparse.py` → `validate.py` → `pipeline.py`. The parser version (`tt-grid-parser/1.2.0`) is stored with every entry. *Tests:* 24 extraction tests and 14 regression tests on the real PDF.

M4. Worker (`app/worker.py`). Claims jobs (Listing 1), runs the pipeline, persists entries, computes the timetable status, notifies the uploader, and retries unexpected failures. It can run as a separate process (`python -m app.worker`) or as a thread inside the API on the free tier.

M5. Query planner (`app/query`). Files `nlp.py`, `vocab.py`, `planner.py`, `composer.py`, `registry.py`, `engine.py` and `templates.py`. Implements Algorithms 4 and 5. `planner.py` turns the parsed question into a typed `QueryPlan`; `composer.py` compiles composed plans (count, exists, total hours, first, last, next, per-day summaries, lab/lecture and before/after/between filters) from the shared building blocks of `templates.py`, always starting from the role-scoped base query. `registry.py` declares the specialised operations (Q01–Q30, templates QT01–QT22) and the composed operation (Q31, QT23) with their domains, parameters and plan vocabulary. Every answer carries its plan in `meta.plan`, so the interpretation is visible. *Tests:* 59 tests in `test_query_engine.py` (including SQL-injection text and unknown parameters) and 7 composed-plan tests on the real timetable in `test_college_seed.py`.

M6. Conflicts (`services/conflicts.py`). Implements Algorithm 3 and returns CONFIRMED/POSSIBLE conflicts, scoped to the HOD's department. *Test:* the real-PDF test asserts that exactly the two genuine clashes are reported.

M7. Academics and administration. Files `services/academics.py`, `users.py` and `api/v1/admin.py`. Fresher registration without duplicates, promotion with preview and confirm (all or nothing, history preserved), and master data CRUD with validation. *Tests:* 13 in `test_academics_admin.py`, including concurrent promotions.

M8. Mobile and web client (`mobile/src`). Described in Section 7.4. *Tests:* 18 Jest tests and Playwright end-to-end tests.

7.4 Front-End Implementation

The app has 26 screens organised with Expo Router (file-based routing):

- **Auth stack:** welcome, login, account activation, forgot password.
- **Tabs:** Home (role dashboard), Ask (NL search), Timetable (week strip and day timeline), Archives, Admin (Admins only; hiding the tab is cosmetic because the server enforces access), Profile.
- **Stack screens:** class details, timetable details (extraction summary, sections), review & correct, upload, conflicts, monitoring, notifications, admin tools (users, new user, promotion, master data, institution settings, jobs, audit log).

Forms use React Hook Form with Zod schemas (e.g. e-mail format, password rules, room code). **Navigation** is file-based. **State**: server data in TanStack Query with keys per domain and timetable, invalidated when the official timetable changes. A **College/Personal toggle** switches the domain, with a different accent colour so users always know which data they see. **Accessibility**: light/dark themes, large tap targets, labels for screen readers. On a wide laptop screen the app is shown as a centred column. The web session is kept in `sessionStorage` (cleared when the tab closes), and on phones in the Keychain/Keystore.

7.5 Back-End Implementation

- **76 REST endpoints** in six routers: `auth`, `me`, `timetables`, `search`, `notifications`, `admin`. The OpenAPI specification is generated automatically (`docs/openapi.json`).
- **Request processing:** middleware adds a request id and JSON logging; dependencies authenticate and load the principal; Pydantic validates bodies; services hold business logic; the error handler converts exceptions to the envelope.
- **Search context:** `services/context.py` decides the domain (request hint → remembered → institutional), the timetable (explicit archive → remembered archive → primary), verifies ownership and usability, and computes "now" in the user's time zone.

- **Static web app:** in the deployed image the same FastAPI process serves the exported web app, so phone and laptop use one HTTPS origin.

7.6 Database Implementation

31 tables are created by Alembic migrations. Templates are SQLAlchemy Core/ORM expressions with bound parameters. A representative template (QT08, next class) is:

Listing 2: Template QT08 (simplified): next class within a scope

```
stmt = (select(E)
        .where(E.timetable_id == ctx.timetable_id,
               E.entry_kind == "CLASS",
               E.verification_status != "REJECTED",
               E.time_uncertain.is_(False),
               or_(E.class_date == day, and_(E.class_date.is_(None),
                                             E.day_of_week == day.isoweekday())),
               E.start_time > after,
               scope_filter(ctx))          # batch + parent batch, or staff id
        .order_by(E.start_time)
        .limit(1))
```

Connectivity: one SQLAlchemy engine per process with a connection pool (`psycopg` 3 driver); each request gets a session that commits or rolls back as one transaction.

7.7 Communication and Concurrency Implementation

- **Client–server:** `mobile/src/api/client.ts` adds the bearer token, applies timeouts, retries idempotent GETs, and performs a *single-flight* refresh: the first 401 starts one refresh call and all other failing requests wait for the same promise, then retry.
- **Concurrent clients:** several Uvicorn processes; each request is independent (stateless), with its own database transaction.
- **Synchronisation:** activation and preference changes use `SELECT ... FOR UPDATE`; job claiming uses `SKIP LOCKED`; uploads are idempotent through a partial unique index; the rate limiter uses a lock-protected in-process sliding window.
- **Recovery:** job heartbeat and re-claim; maximum attempts; the client shows the cached institutional timetable with “last refreshed” time when offline.

7.8 Git Repository Setup

- **Repository URL:** `https://github.com/shivammulani-24/Dynamic_Timetable`
- **Structure:** `backend/` (app, alembic, tests), `mobile/` (src, ___tests___, e2e), `docs/` (13 documents + OpenAPI), `deploy/` (Render image), `scripts/` (dev script, Windows one-click start, load smoke), `report/` (this report and poster), `render.yaml`, `docker-compose.yml`.
- **Branching strategy:** `main` is the stable branch; development happens on a feature branch that is merged through a pull request after review. The Render demo deploys from the development branch, so every push updates the demo.
- **Commit message convention:** an imperative summary of what changed and why, naming the scope (e.g. "Query engine: intent registry Q01–Q30 ..."), with the details in the body.

Table 7.1: Development history (commits on the development branch)

Commit	Date	Message (summary)	Files / lines
`0dfda92`	2026-10-09	Backend foundation: schema, migrations, auth, RBAC, admin master data	46 / +5164
`bc917fb`	2026-10-09	Extraction engine (PDF/OCR/Excel/CSV/Word/images), worker, archives, review, conflicts	30 / +3841
`47e3d89`	2026-10-09	Query engine: intent registry Q01–Q30, templates QT01–QT22, entity resolution, clarification	17 / +3026
`1aa651d`	2026-10-09	Expo mobile app: role dashboards, NL search, weekly timetable, archives, uploads, review, admin; Jest + Playwright	72 / +18618
`f028feb`	2026-10-09	Academic/admin/conflict/concurrency tests, batched vocabulary loading, load smoke script	7 / +353
`4f3528b`	2026-10-09	Documentation, OpenAPI export, Docker/compose, EAS profiles, pinned requirements	23 / +7345
`bb81091`	2026-10-09	Tune extraction on the college's real timetable PDF (parser 1.1.0)	13 / +508
`dc12b25`	2026-10-09	Apply the college's timetable rules; real-PDF seed; phone guide	31 / +1106
`710e679`	2026-10-14	College seed: one student login per lab group	3 / +24

Commit	Date	Message (summary)	Files / lines
`fce3644`	2026-10-21	Run the app in a laptop browser; one-click Windows start	7 / +129
`e8f74c3`	2026-10-21	Hide admin review notices from students; fix archive banner	12 / +102
`326b8e6`	2026-10-25	Upgrade the app to Expo SDK 58	8 / +9018 (lockfile)
`8093f09`	2026-10-25	Free online demo (Render) + fix account takeover via password reset (P0-1)	25 / +444
`ccfb97d`	2026-10-27	Mini project report (LaTeX), A3 poster and ESE demo guide	report
latest	2026-10-31	Compositional query planner (query plans, composed executor QT23, app result cards, tests)	see Git log

Contribution of individual group members. Work was divided by module so that each member owned one major part end to end, while requirements, testing and reviews were shared.

Table 7.2: Contribution of group members

Member	Main contributions
Shivam Mulani (2024300141)	System architecture and database design (schema, migrations, indexes); authentication and role-based access control; natural-language query engine (intent registry Q01–Q30, templates QT01–QT22, clarifications); concurrency control; security audit and fixes; cloud deployment on Render.
Gurshaan Nandrajog (2024300146)	Timetable extraction pipeline (PDF tables, OCR, Excel/Word adapters); tuning on the college's real PDF (lab groups, legends, noon row, combined B.E. page); background worker; conflict detection; extraction and regression tests; performance and load measurements.
Harshvarddhan Waghela (2024300272)	Cross-platform app (React Native/Expo): role dashboards, weekly view, Ask screen, upload, review and admin screens; laptop/web support and Expo SDK upgrade; mobile and end-to-end tests; documentation, report and poster.
All members	Requirements analysis and SRS, design reviews with the guide, testing on phones and laptops, demo preparation.

7.9 Integration of Modules

Modules communicate only through defined interfaces: the app uses the REST API; routers call services; services call the query engine or put jobs in the queue; the worker calls the extraction pipeline and writes through the same models. Integration was verified at three levels: (1) back-end integration tests that upload a file, run the worker, activate the timetable and query it through the API; (2) the college seed test, which runs the real PDF through the whole pipeline and then asks questions as students, faculty, HOD and Admin; (3) Playwright end-to-end tests that drive the real UI against the real API, including uploading a PDF through the file picker and waiting for the worker.

7.10 Implementation Challenges and Resolutions

Table 7.3: Implementation challenges and resolutions

Challenge	Cause	Resolution
Noon row printed as "12.15 a.m. to 01.15 p.m."	Typing error in the source PDF	Accept midday only when proven by both neighbouring rows (Algorithm 1)
Stacked lab cells (A1–A4 in one cell)	Four sub-groups share one grid cell	Line grouping by sub-group token; each becomes its own entry with its own room/teacher
Legends split across columns, codes missing from legend	Page layout	Page-local legend parser; unknown codes kept as written, with an INFO note (not an error)
Combined B.E. electives across divisions A–D	One page serves four divisions	Map to a common parent batch so all B.E. divisions see it
LLC slots with no teacher or room	Co-curricular course	Treat as a valid class without teacher/room; app hides empty fields
542 raw overlaps	Parallel labs and electives are intended	Intended-overlap rules → 2 genuine clashes
Slow searches (about 50 SQL statements)	N+1 lazy loading of related rows	Eager loading and batched vocabulary → about 25

Challenge	Cause	Resolution
Web app stuck on "Loading…" in production	Connectivity probe used `HEAD /`, server returned 405	Accept `GET` and `HEAD` on web routes; regression test
Account takeover risk (P0-1)	Password-reset code could be returned/logged in non-dev configuration	Never return or log codes in production; 9 verification checks
Expo Go on iPhone required SDK 58	Store Expo Go supports only the newest SDK	Upgraded to SDK 58 / RN 0.88; fixed type changes; mocked icon fonts in Jest
Free hosting allows one service	Render free tier	In-process worker thread using the same `SKIP LOCKED` claim; static web app served by FastAPI
PowerShell blocked `npx` on Windows	Execution policy	Documented `npx.cmd` / policy change; one-click `start-windows.bat`

Chapter 8: Scalability, Performance and Resource Management

Tested vs. projected. All numbers in this chapter labelled *measured* were collected by us on the development machine described in Section 8.6, with the real college timetable loaded. Numbers labelled *projected* are estimates and have not been tested on production hardware.

8.1 Scalability Requirements

Table 8.1: Expected scale of one institute (projected)

Quantity	Expected value
Registered users (students + staff)	2,000–3,000
Batches / lab groups	50–100 (real PDF: 46)
Class entries per official timetable	500–1,000 (real PDF: 453 classes, 570 rows)
Timetable versions per semester	3–10
Peak concurrent users (e.g. just after a timetable change)	50–200
Peak request rate	20–60 requests/s (a few requests per active user per minute)
Uploads	a few per week; each $\leq$ 15 MB, $\leq$ 30 pages

8.2 Workload and Load Model

The load test simulates students using the app at the same time:

- $N \in \{1, 2, 4, 8, 16, 32\}$ concurrent clients (threads), each logged in as a *different* real student account (37 accounts, one per lab group).
- Each client sends 30 search requests back-to-back with no think time (closed-loop, worst case), cycling through five realistic questions: “What is my next class?”, “What are my classes on Monday?”, “Do I have classes after lunch on Tuesday?”, “Which rooms are free at 10 am today?” and “Find all DBMS classes”.
- Total requests per run $= 30N$ (30 to 960).
- Every response is checked: HTTP 200 with `OK`/`NO_MATCH`, or the expected 422 clarification. (The M.Tech. page has no lunch row, so “after lunch” correctly asks for a time.)

8.3 Resource Utilisation (measured)

Table 8.2: Measured resource utilisation

Resource	Value	How measured
Memory per API worker process (after load test)	113–116 MB	`ps -o rss`, 4 workers
Peak memory, extraction of the 10-page PDF	82 MB	`getrusage` max RSS
Database size (one timetable, 91 users, history)	12 MB	`pg_database_size`
Entry table incl. indexes (570 rows)	592 kB	`pg_total_relation_size`
Original PDF stored	420 kB	file size
CPU during 32-client test	≈76% of 4 cores	process CPU time / wall time
Web app bundle (JavaScript)	3.5 MB uncompressed	exported bundle size
Network per search	≈1–6 kB JSON	response size (depends on result count)

The KPI targets were under 300 MB per API worker and under 50 MB of database; both are met with a large margin.

8.4 Concurrent Processing

Requests are handled by several independent Uvicorn worker *processes* (4 in the test), each with its own connection pool. Because the API is stateless, a request can go to any process. Within a process, FastAPI runs synchronous endpoints in a thread pool, so slow database calls do not block other requests. Extraction never runs in an API request: it is a job processed by the worker, so a large OCR job cannot slow down searches. Shared state lives only in PostgreSQL and is protected by row locks (Section 7.7).

8.5 Scheduling and Synchronisation Analysis

- **Request scheduling** is done by the operating system (processes) and Uvicorn's event loop. Under overload, requests queue at the socket. The rising p50 latency beyond 8 clients (Table 8.4) is this queueing delay, since throughput stays flat at the CPU limit.
- **Job scheduling** is FIFO by `created_at`. Jobs are short (about 1.5 s for the real PDF), so FIFO gives low waiting times and needs no pre-emption. Round-Robin would only help

with long jobs that need to share a worker, which our workload does not have.

- **Synchronisation measurements:** the concurrency tests start 2 activations and 2 promotions at the same moment. In every run exactly one wins and the other sees the committed result, with no deadlock and no lost update. The load test reported 0 inconsistent answers.

8.6 Performance Measurement

- **Environment:** Linux VM, 4 vCPU, 15 GB RAM; PostgreSQL 16 on the same machine; API with 4 Uvicorn workers; the load generator also on the same machine (which takes CPU from the server, so results are conservative).
- **Data:** the college's real timetable (453 classes), 46 batches, 52 faculty, 42 rooms, 91 user accounts.
- **Tools:** a Python script with `httpx` and threads; `time.perf_counter` timestamps around each request; percentiles computed from all samples; `ps` for CPU and memory. For the load test only, the per-user rate limits were raised, because otherwise the limiter (60 searches/min per user) correctly rejects the synthetic burst.
- **Single-user latency:** 50 sequential requests per question type after one warm-up request.

8.7 Performance Results (measured)

Table 8.3: Single-user search latency per question type (50 requests each)

Intent	Question	Results	p50 (ms)	p95 (ms)
Q10 Next class	What is my next class?	1	42	52
Q02 Day timetable	What are my classes on Monday?	5	44	52
Q06 Course search	Find all DBMS classes	7	45	51
Q11 After lunch	Do I have classes after lunch on Tuesday?	3	47	54
Q15 Professor schedule	What is my schedule on Monday?	3	37	46
Q22 Free rooms	Which rooms are free at 10 am today?	20	43	51
Q20 Room schedule	Show the schedule of room 508 on Monday	8	41	49

Table 8.4: Load test with increasing concurrent clients (4 API workers)

Clients	Requests	Wrong	req/s	p50 (ms)	p95 (ms)	CPU (%)
1	30	0	18.6	49	57	31
2	60	0	25.4	75	90	32
4	120	0	46.5	64	90	48
8	240	0	60.4	90	141	63
16	480	0	54.1	183	354	62
32	960	0	57.4	410	635	76

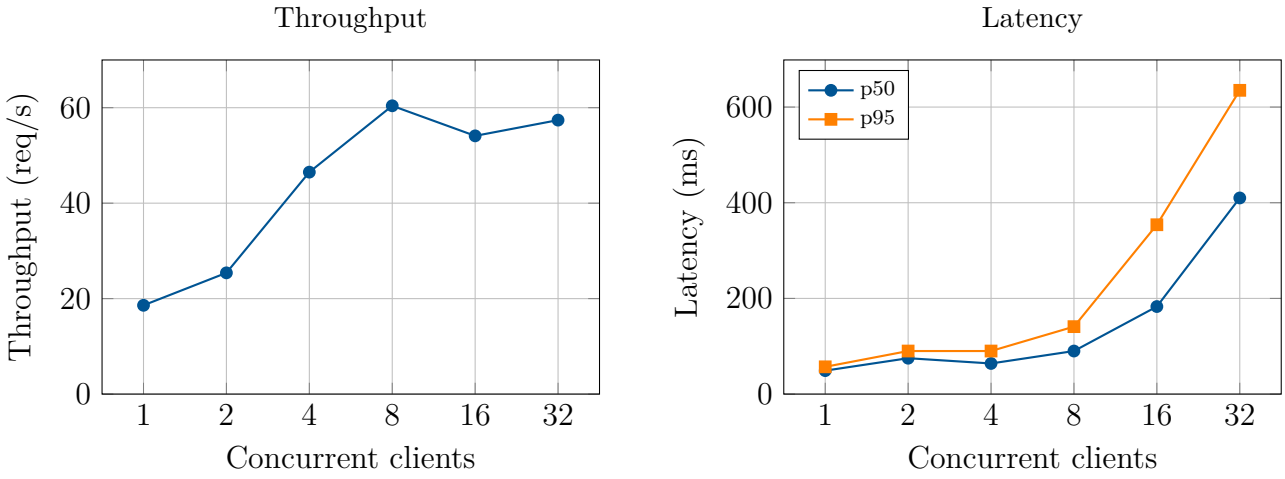


Figure 8.1: Throughput and latency under increasing concurrent load (measured)

Table 8.5: Extraction performance on the real 10-page PDF (median of 5 runs)

Metric	Value
Pages / rows produced	10 / 570 (453 classes)
Extraction time, whole pipeline	1.65 s (runs: 1.72, 1.65, 1.64, 1.69, 1.64 s)
Per page	≈0.17 s
Peak memory	82 MB

8.8 Bottleneck Identification

1. **CPU of the API processes.** Throughput stops growing at about 55–60 req/s from 8 clients while CPU reaches 60–76%, and the rest of the CPU is taken by PostgreSQL and the load generator on the same machine. Beyond that point extra clients only add queueing delay, so p50 rises roughly linearly with N.

2. **Per-request work:** each search resolves the context, loads the vocabulary of the timetable (about 200 labels) and runs one or two templates. Vocabulary loading is the largest repeated cost.
3. **Database round trips:** before optimisation a search issued about 50 SQL statements because related rows were loaded lazily (N+1 problem).

8.9 Optimisation and Improvement

Table 8.6: Optimisations performed (before/after)

Change	Before	After	Effect
Eager loading of course/staff/room/-batch (N+1 fix)	≈50 SQL statements / search	≈25	Halved round trips; latency now 37–47 ms
Batched vocabulary loading	1 query per label	1 query per entity type	Constant number of queries per search
Composite indexes led by `timetable_id`	–	index range scans	"Now/next" independent of other timetables' size
Intended-overlap rules in conflict detection	542 reported overlaps	2	Removes false alarms (quality, not speed)
Extraction moved to background worker	–	API never blocked by OCR	Search latency unaffected by uploads

Further options: cache the vocabulary per (timetable, `data_version`); add more API processes or machines (horizontal scaling); use a shared rate-limit store (Redis/PostgreSQL) for several machines.

8.10 Cloud Deployment and Resource Utilisation

- **Architecture:** Figure 4.9. One Render *free* web service runs a Docker image (multi-stage build: Node builds the web app, then a Python image with Tesseract runs FastAPI, the worker thread and the static web app), plus a free managed PostgreSQL database.
- **Configuration:** `render.yaml` (blueprint); the secret key is generated by Render, `DEMO_PASSWORD` is entered at deploy time, production mode refuses the published development password, and demo data is seeded only into an empty database.
- **Resources:** free instance with 512 MB RAM and a shared fraction of a CPU. Measured memory of one API process (≈115 MB) plus extraction peak (≈82 MB) fits within it.

- **Limits:** the service sleeps after 15 minutes idle (first request then takes about a minute); the free database expires after 30 days; 750 free instance hours per month [34].
- **Scaling approach (projected):** a paid always-on instance with 2+ API processes and a separate worker; managed PostgreSQL with backups; horizontal scaling behind the platform's load balancer, because the API is stateless.

8.11 Scalability Limitations

- **Maximum tested workload:** 32 concurrent closed-loop clients (960 requests) with 0 wrong answers and 57 req/s, on a 4-core development VM shared with the database and the load generator.
- **Projection (not tested):** with think time of about 10 s between requests, 57 req/s corresponds to several hundred simultaneously active users on similar hardware. This is an estimate only; a real capacity test must run on the deployment hardware.
- The free cloud tier is much smaller (shared CPU); it is suitable for a demo, not for the whole college.
- The rate limiter is per process. With several machines it must move to a shared store.
- OCR of large scanned PDFs is CPU-heavy; with many simultaneous uploads more workers are needed.

Chapter 9: Testing and Validation

9.1 Testing Strategy

Testing followed the test pyramid: many fast unit tests, fewer integration tests, and a small number of end-to-end tests that drive the real user interface. Tests were written alongside each module and the full suite was run before every commit. The real college PDF is part of the test fixtures, so every change to the parser is checked against the real document (regression testing).

Table 9.1: Test suites and results (latest run)

Suite (file)	Tests	What it covers
`test_auth.py`	15	Login, tokens, rotation, reuse detection, suspension, role change, rate limit, reset-code safety
`test_extraction.py`	24	PDF-01…PDF-13 of the extraction specification, OCR, other formats, time rules, cell grammar
`test_reference_pdf.py`	14	Regression on the real college PDF (10 pages)
`test_college_seed.py`	28	Real PDF end to end: seed → API as student, professor, HOD, Admin; 7 composed query-plan tests (count, first/last, busiest day, lab filter scope, exists, counting professors/subjects, clarification, injection)
`test_uploads_and_domains.py`	13	Upload → worker → status, primary rules, IDOR, idempotency, corrections
`test_query_engine.py`	59	Every specialised operation Q01–Q30, composed plans in the personal domain, clarifications, domain isolation, injection text
`test_academics_admin.py`	13	Registration, promotion, concurrency, master data, role matrix, conflicts
`test_deploy.py`	8	Web app serving, `HEAD` connectivity check, production safety
Back end (pytest)	**174**	**174 passed, 0 failed**
Mobile unit/component (Jest)	20	API client, single-flight refresh, offline cache, result renderer (incl. count and day-summary cards), status labels
End-to-end (Playwright)	24 + 21	Real college timetable flow 24/24 (incl. composed questions); synthetic smoke flow incl. upload 21/21
TypeScript `strict` type check	–	clean

9.2 Unit Testing

Unit tests check individual functions with exact inputs and outputs. Examples: `test_time_rules` (time-label parsing and sequence resolution, including "12.15 a.m. to 01.15 p.m."), `test_cell_grammar_explicit_time_and_electives` (cell parser), `test_signature_detection_rejects_mismatch` (plain text named `x.pdf` is rejected; a real PDF is recognised even when renamed), and, in Jest, the single-flight token refresh (three concurrent 401 responses produce exactly one refresh call).

9.3 Integration Testing

Integration tests use a real PostgreSQL test database and FastAPI's test client: `test_personal_upload_processing_and_primary_unchanged` uploads a file, runs the worker and checks the status transitions; `test_real_pdf_end_to_end_upload_persist_activate` runs the real PDF through upload, worker, database and activation; `test_college_seed.py` loads the real timetable and asks questions as different roles. Playwright then tests the front end, back end and database together in a browser.

9.4 Functional Testing

Each functional requirement has at least one test (Table 9.2). The intent tests are organised one per intent (`test_q01_...` to `test_q30_...`), each with valid and negative paths.

9.5 Non-Functional Testing

- **Performance:** single-user latency and load tests (Chapter 8).
- **Security:** role escalation (`test_role_escalation_not_possible_via_client`), IDOR (`test_idor_other_users_personal_timetable`), SQL-injection text (`test_sql_injection_text_i`
  raw SQL parameters rejected, rate limiting, token reuse detection, reset codes never logged in production. A manual security audit (OWASP categories) found one critical issue (P0-1), which was fixed and verified with 9 checks.
- **Reliability:** worker retry, crash recovery by heartbeat, idempotent uploads.
- **Usability:** the app was tried on Android and iPhone (Expo Go) and laptop browsers by the team. This hands-on use led to hiding Admin review notices from students and fixing the archive banner.
- **Portability:** Android and iOS bundles compile; the web build runs in Chrome; the Docker image deploys on Render.

9.6 Boundary and Invalid-Input Testing

- Empty, corrupt and wrong-type files; a PDF with no table; a 0-entry result (UNUSABLE).
- Time boundaries: back-to-back classes (9–10 and 10–11) do *not* conflict (end-exclusive); a class at exactly its start time is "current".
- Ambiguous inputs: "Am I free at 2?" (a.m./p.m.), "5/6" (date order), an unknown professor name, a misspelled name (suggestions only).
- Date range larger than 31 days rejected; unknown parameters and raw SQL rejected.
- Invalid state transitions: an UNUSABLE or FAILED timetable cannot become primary; a student cannot be promoted twice; an Admin cannot remove their own Admin role.
- Missing configuration: no lunch boundary, no working hours, no week start → clarification or explicit "not configured", never an assumed value.

9.7 Concurrency and Stress Testing

- `test_concurrent_activation_single_default`: two threads activate different timetables at the same time; exactly one default results, with no lost update.
- `test_concurrent_promotions_cannot_create_two_open_placements`: simultaneous promotions of the same student; the partial unique index and row locks guarantee one open placement.
- Single-flight refresh under concurrent 401s (Jest).
- Load test up to 32 concurrent clients: 960 requests, 0 wrong answers, no deadlocks or errors (Section 8.7).

9.8 Test Case Specification

Table 9.2: Test case specification

TC	Req.	Scenario (input)	Expected result	Actual	P/F	Evidence
TC-01	FR-01	Wrong password vs unknown user (Two bad logins)	Same generic error	As expected	P	test_auth
TC-02	FR-01	Refresh-token reuse (Reuse rotated token)	Whole family revoked, 401	As expected	P	test_auth
TC-03	FR-02	Role escalation (Student sends `roles:[ADMIN]`)	Ignored; admin APIs 403	As expected	P	test_auth

TC	Req.	Scenario (input)	Expected result	Actual	P/F	Evidence
TC-04	FR-03	Wrong file type (Text file named `.pdf`)	Rejected, nothing stored	As expected	P	test_ extraction
TC-05	FR-03	Upload does not change primary (Upload new file)	Primary unchanged	As expected	P	test_uploads
TC-06	FR-03	Retry same upload (Same client_request_id twice)	One archive record	As expected	P	test_uploads
TC-07	FR-04	Real PDF extraction (10-page college PDF)	≥95% verified	452/453 (99.8%)	P	test_college_ seed
TC-08	FR-04	Stacked lab cell (Cell with A1–A4)	4 entries, own room/teacher	As expected	P	test_ reference_pdf
TC-09	FR-04	Noon row typo (“12.15 a.m. to 01.15 p.m.”)	12:15–13:15 with visible note	As expected	P	test_ reference_pdf
TC-10	FR-04	Scanned PDF (Image-only PDF)	OCR, entries UNVERIFIED	As expected	P	test_ extraction
TC-11	FR-05	No-table PDF (Plain text PDF)	UNUSABLE, cannot be primary	As expected	P	test_ extraction
TC-12	FR-06	Correction audited (Admin edits room)	Original kept, audit row	As expected	P	test_uploads
TC-13	FR-07	Concurrent activation (2 threads activate)	Exactly one default	As expected	P	test_ academics
TC-14	FR-09	Dashboard = query (Home vs “next class?”)	Identical entry	As expected	P	test_query_ engine
TC-15	FR-10	Lab group isolation (SE-A1, Wednesday)	Only A1 labs	As expected	P	test_college_ seed
TC-16	FR-11	Lunch per year (SE vs TE “after lunch”)	14:15 vs 13:15 from PDF	As expected	P	test_college_ seed
TC-17	FR-11	Ambiguous time (“Am I free at 2?”)	Clarification a.m./p.m.	As expected	P	test_query_ engine, e2e
TC-18	FR-11	Injection text (“’; DROP TABLE . . . ”)	Treated as text; no SQL run	As expected	P	test_query_ engine
TC-19	FR-11	Cross-domain compare (Q30 question)	UNSUPPORTED, no query executed	As expected	P	test_query_ engine
TC-20	FR-12	Free rooms on a floor (“free rooms on floor 5 at 10”)	Rooms without overlapping class	As expected	P	test_query_ engine
TC-21	FR-13	Clashes in real PDF (Real timetable)	Exactly 2 genuine clashes	2 (SP Fri; 603-2 Tue)	P	test_college_ seed
TC-22	FR-14	IDOR personal (Other user’s timetable id)	404 + audit	As expected	P	test_uploads
TC-23	FR-15	Promotion (Preview + confirm)	History kept, new batch	As expected	P	test_ academics

TC	Req.	Scenario (input)	Expected result	Actual	P/F	Evidence
TC-24	FR-15	Concurrent promotion (2 threads)	One open placement	As expected	P	test_ academics
TC-25	FR-17	Student without batch (“next class?”)	Setup-required message	As expected	P	test_college_ seed
TC-26	NFR-04	Reset code in prod (Production, no SMTP)	Code never in response/log	As expected	P	test_deploy
TC-27	NFR-03	Connectivity probe (`HEAD /`)	200 (not 405)	As expected	P	test_deploy
TC-28	NFR-01	Search latency (50 req/intent)	p50 $<$ 300 ms	37–47 ms	P	Table 8.3
TC-29	NFR-03	Load (32 clients, 960 req)	0 wrong answers	0	P	Table 8.4
TC-30	UI	Full student/HOD flow (Browser, real data)	All steps pass	24/24	P	college.e2e.mjs
TC-31	FR-11	Count with filters (“How many labs do I have this week?”)	Count = distinct lab periods; only labs	As expected	P	test_college_ seed
TC-32	FR-11	First/last (“When is my last class on Friday?”)	Same as max end of the Friday listing	As expected	P	test_college_ seed
TC-33	FR-11	Day-wise ordering (“Which day is my busiest / finish earliest?”)	Days sorted by classes / finish time	As expected	P	test_college_ seed
TC-34	FR-02	Composed plan keeps role scope (“Show my labs”, “Show SE-B labs”)	Only own lab group; no other batch	As expected	P	test_college_ seed

9.9 Defect Identification and Resolution

Table 9.3: Significant defects found, fixed and re-tested

ID	Defect	Root cause and fix	Re-test
D-1	542 "conflicts" in real PDF	Parallel labs/electives treated as clashes; added intended-overlap rules	2 remain; test asserts exact set
D-2	Noon classes missing	"12.15 a.m." rejected as invalid; added neighbour-proof rule	Regression test
D-3	Unknown legend codes shown as errors	Codes not in legend are valid; downgraded to INFO	Test
D-4	Students saw "needs review" warnings	Admin-only notice leaked to students; filtered by role	Test + screenshot
D-5	Archive banner shown when choosing the official timetable	Explicit selection of primary treated as archive	Test
D-6	Web app stuck on "Loading…"	`HEAD /` returned 405 ⇒ "offline"; allow HEAD	Test + e2e
D-7	Account takeover via reset (P0-1)	Reset code exposed outside development; never returned/logged in production	9 checks + test
D-8	≈50 SQL statements per search	N+1 lazy loading; eager loading	≈25; latency measured
D-9	App incompatible with Expo Go on iPhone	SDK 57 vs required 58; upgraded, fixed types and Jest mocks	All tests + e2e

9.10 Validation Against KPIs

Table 9.4: KPI targets (Chapter 1) vs measured results

KPI	Target	Measured	Met?
Extraction verification rate	≥95%	452/453 = 99.8% (remaining entry has no time in the PDF)	Yes
False clash reports	0	0 (2 reported, both genuine)	Yes
Response time (median, 1 user)	<300 ms	37–47 ms	Yes
Throughput (4 workers)	≥30 req/s	46.5 req/s at 4 clients, 60.4 at 8	Yes
Reliability under concurrency	100%	960/960 correct at 32 clients	Yes
Functional correctness	100% tests pass	174/174, 20/20, 24/24, 21/21	Yes
Extraction time (10 pages)	<10 s	1.65 s	Yes
Memory per API worker; DB size	<300 MB; <50 MB	≈115 MB; 12 MB	Yes
Supported question types	≥25	30 specialised operations + composed plans (8 operations × filters × windows)	Yes

9.11 Limitations and Untested Scenarios

- **Extraction accuracy** has not been measured against a hand-labelled ground truth (precision/recall). The 99.8% figure is the share of entries the system *verified itself*; the 2 clashes were confirmed by hand.
- **Capacity on production hardware** has not been tested; the load test ran on a development VM.
- **Push notifications** (need a store build), **SMTP e-mail** and **store builds** (APK/TestFlight) have not been exercised.
- **Device matrix:** tried on a few phones through Expo Go and the web; no systematic small-screen/large-text/accessibility audit.
- **Other colleges' timetable layouts** have not been tested; only our department's PDF and synthetic fixtures.

- **Long-running operation** (database expiry, backups, multi-semester data) has not been tested beyond the 30-day free-tier window.

Chapter 10: Results, Demonstration and Discussion

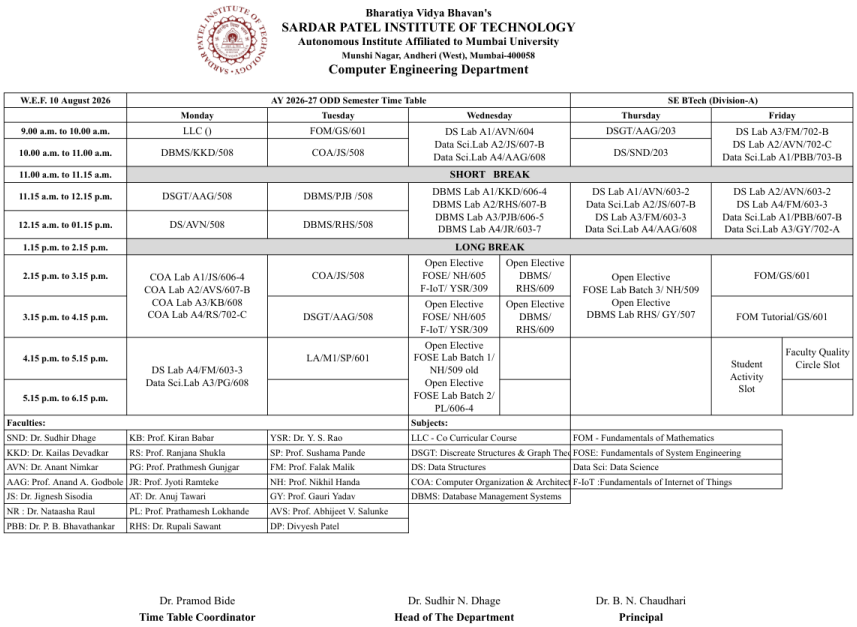
10.1 Implemented Features

Table 10.1: Implementation status of features (implemented = code + automated test)

Feature	Status	Evidence
Login, activation, password reset, 5 roles with server-side RBAC	Implemented, tested	test_auth (15)
Upload PDF / scanned PDF / image / XLSX / XLS / CSV / DOCX / DOC	Implemented, tested	test_extraction (24)
Extraction tuned on real college PDF (labs, legends, noon row, BE combined, LLC)	Implemented, tested	test_reference_pdf (14)
Validation, review & correction (audited), activation, archives	Implemented, tested	test_uploads (13)
Conflict detection (professor / room / batch)	Implemented, tested	2 genuine clashes found
Role dashboards, weekly view, class details	Implemented, e2e tested	Figures 10.2–10.4
Natural-language search: compositional query plans (count, first/last, per day, lab/lecture, before/after) plus specialised operations, clarifications	Implemented, tested	test_query_engine (59), composed tests (7)
Personal timetables, strictly separate	Implemented, tested	domain tests
Promotion, fresher registration, master data, users, audit log	Implemented, tested	test_academics (13)
Android, iPhone (Expo Go), laptop browser	Implemented; web e2e; device tried	Screenshots
Free online demo (Render)	Deployed	`render.yaml`, demo link
Push notifications	Implemented, not verified	Needs store build
E-mail delivery (SMTP)	Implemented, not verified	No mail server configured
Installable store apps (APK/TestFlight)	Configured (EAS), not built	–

10.2 Screenshots and Demonstration Evidence

The screenshots below were captured automatically by the Playwright end-to-end tests running the real app (web build, phone-sized viewport) against the real API, with the college's timetable loaded.



Bharatiya Vidya Bhavan's
SARDAR PATEL INSTITUTE OF TECHNOLOGY
Autonomous Institute Affiliated to Mumbai University
Munshi Nagar, Andheri (West), Mumbai-400058
Computer Engineering Department

W.E.F. 10 August 2026	AY 2026-27 ODD Semester Time Table				SE BTech (Division-A)		
	Monday	Tuesday	Wednesday		Thursday	Friday	
9.00 a.m. to 10.00 a.m.	LLC ()	FOM/GS/601	DS Lab A1/AVN/604 Data Sci.Lab A2/JS/607-B Data Sci.Lab A4/AAG/608		DSGT/AAG/203	DS Lab A3/FM/702-B DS Lab A2/AVN/702-C Data Sci.Lab A1/PBB/703-B	
10.00 a.m. to 11.00 a.m.	DBMS/KKD/508	COA/JS/508			DS/SND/203		
11.00 a.m. to 11.15 a.m.	SHORT BREAK						
11.15 a.m. to 12.15 p.m.	DSGT/AAG/508	DBMS/PJB /508	DBMS Lab A1/KKD/606-4 DBMS Lab A2/RHS/607-B DBMS Lab A3/PJB/606-5 DBMS Lab A4/JR/603-7		DS Lab A1/AVN/603-2 Data Sci.Lab A2/JS/607-B DS Lab A3/FM/603-3 Data Sci.Lab A4/AAG/608	DS Lab A2/AVN/603-2 DS Lab A4/FM/603-3 Data Sci.Lab A1/PBB/607-B Data Sci.Lab A3/GY/702-A	
12.15 a.m. to 01.15 p.m.	DS/AVN/508	DBMS/RHS/508					
1.15 p.m. to 2.15 p.m.	LONG BREAK						
2.15 p.m. to 3.15 p.m.	COA Lab A1/JS/606-4 COA Lab A2/AVS/607-B COA Lab A3/KB/608 COA Lab A4/RS/702-C	COA/JS/508	Open Elective FOSE/ NH/605 F-IoT/ YSR/309	Open Elective DBMS/ RHS/609	Open Elective FOSE Lab Batch 3/ NH/509 Open Elective DBMS Lab RHS/ GY/507	FOM/GS/601	
3.15 p.m. to 4.15 p.m.		DSGT/AAG/508	Open Elective FOSE/ NH/605 F-IoT/ YSR/309	Open Elective DBMS/ RHS/609		FOM Tutorial/GS/601	
4.15 p.m. to 5.15 p.m.	DS Lab A4/FM/603-3 Data Sci.Lab A3/PG/608	LA/M1/SP/601	Open Elective FOSE Lab Batch 1/ NH/509 old			Student Activity Slot	Faculty Quality Circle Slot
5.15 p.m. to 6.15 p.m.			Open Elective FOSE Lab Batch 2/ PL/606-4				

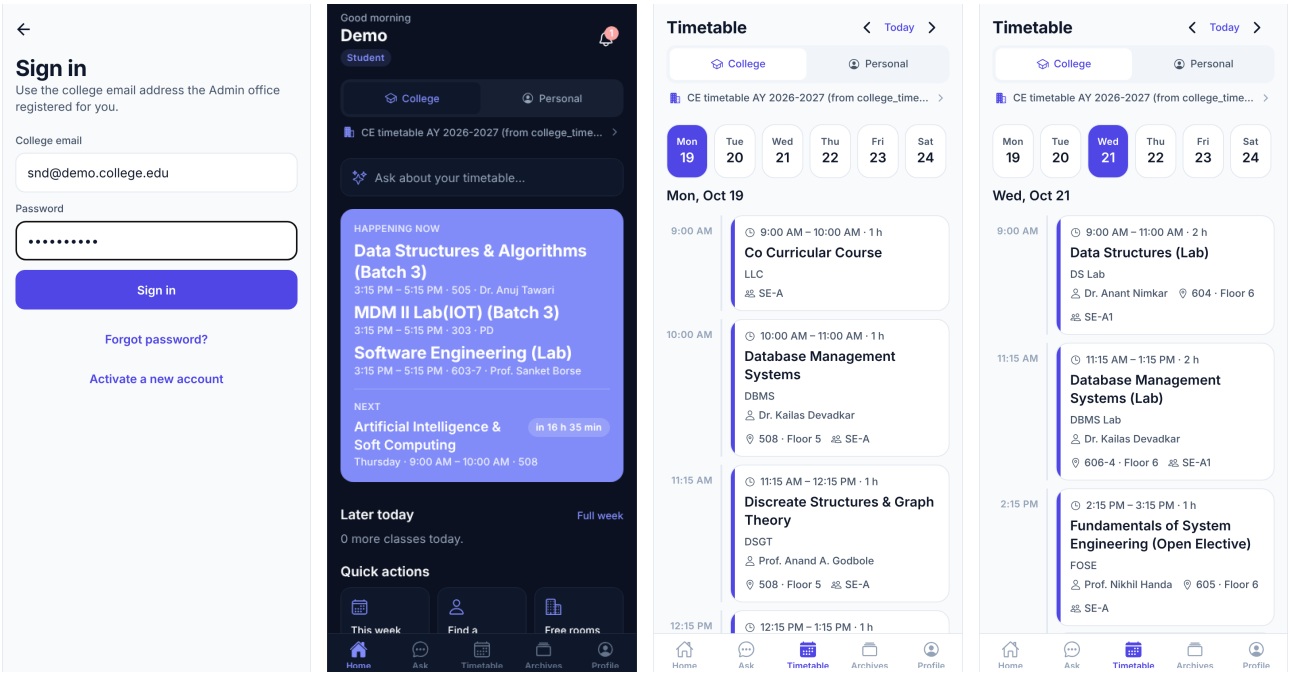
Faculties:			Subjects:	
SND: Dr. Sudhir Dhage	KB: Prof. Kiran Babar	YSR: Dr. Y. S. Rao	LLC - Co Curricular Course	FOM - Fundamentals of Mathematics
KKD: Dr. Kailas Devadkar	RS: Prof. Ranjana Shukla	SP: Prof. Sushama Pande	DSGT: Discreate Structures & Graph The	FOSE: Fundamentals of System Engineering
AVN: Dr. Anant Nimkar	PG: Prof. Prathmesh Gunjgar	FM: Prof. Falak Malik	DS: Data Structures	Data Sci: Data Science
AAG: Prof. Anand A. Godbole	JR: Prof. Jyoti Ramteke	NH: Prof. Nikhil Handa	COA: Computer Organization & Architect	F-IoT :Fundamentals of Internet of Things
JS: Dr. Jignesh Sisodia	AT: Dr. Anuj Tawari	GY: Prof. Gauri Yadav	DBMS: Database Management Systems	
NR : Dr. Nataasha Raul	PL: Prof. Prathamesh Lokhande	AVS: Prof. Abhijeet V. Salunke		
PBB: Dr. P. B. Bhavathankar	RHS: Dr. Rupali Sawant	DP: Divyesh Patel		

Dr. Pramod Bide
Time Table Coordinator

Dr. Sudhir N. Dhage
Head of The Department

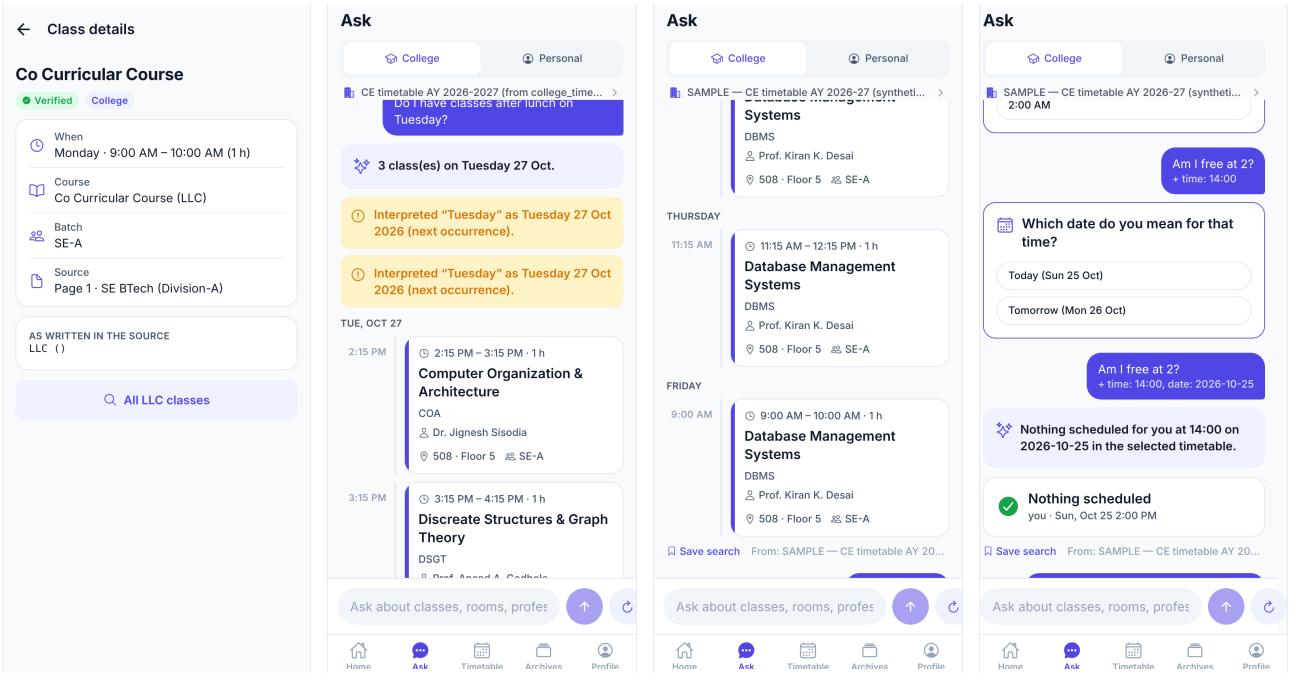
Dr. B. N. Chaudhari
Principal

Figure 10.1: Input: page 1 of the college's real timetable PDF (S.E. Division A), with stacked lab cells and legend



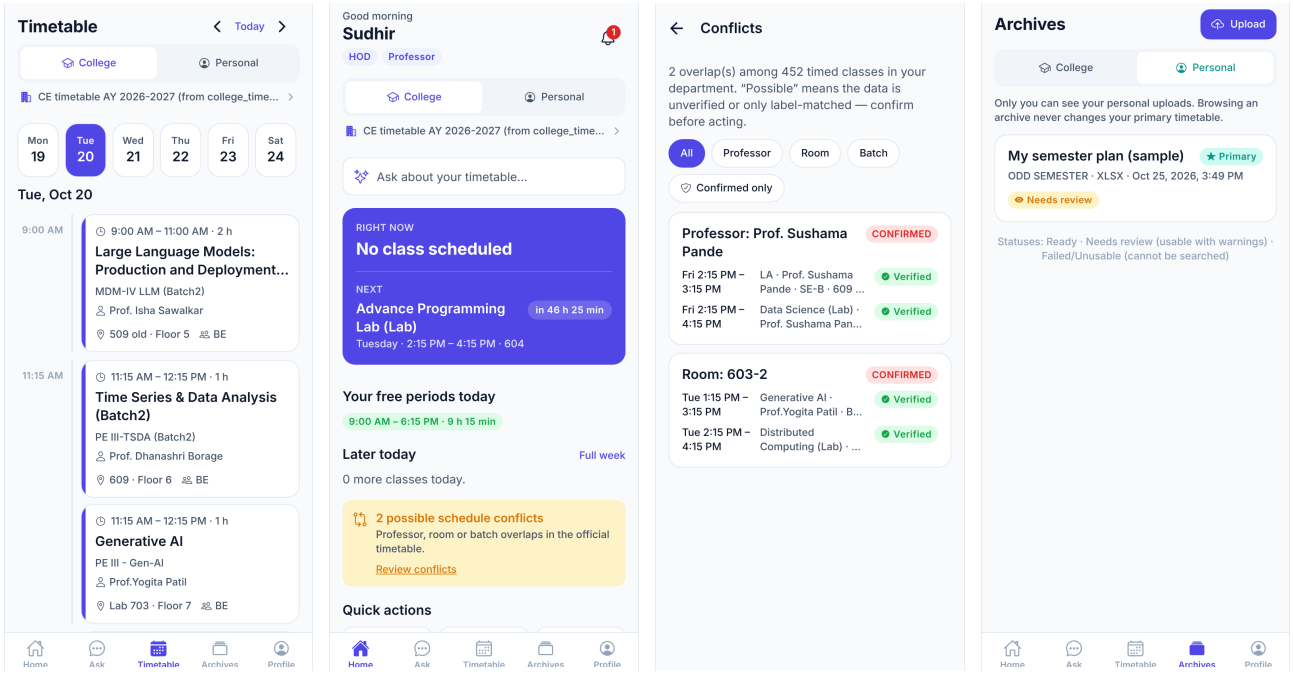
(a) Sign in **(b)** Student home (dark mode): now / next **(c)** Week view, Monday (SE-A1) **(d)** Only lab group A1's labs

Figure 10.2: Student screens



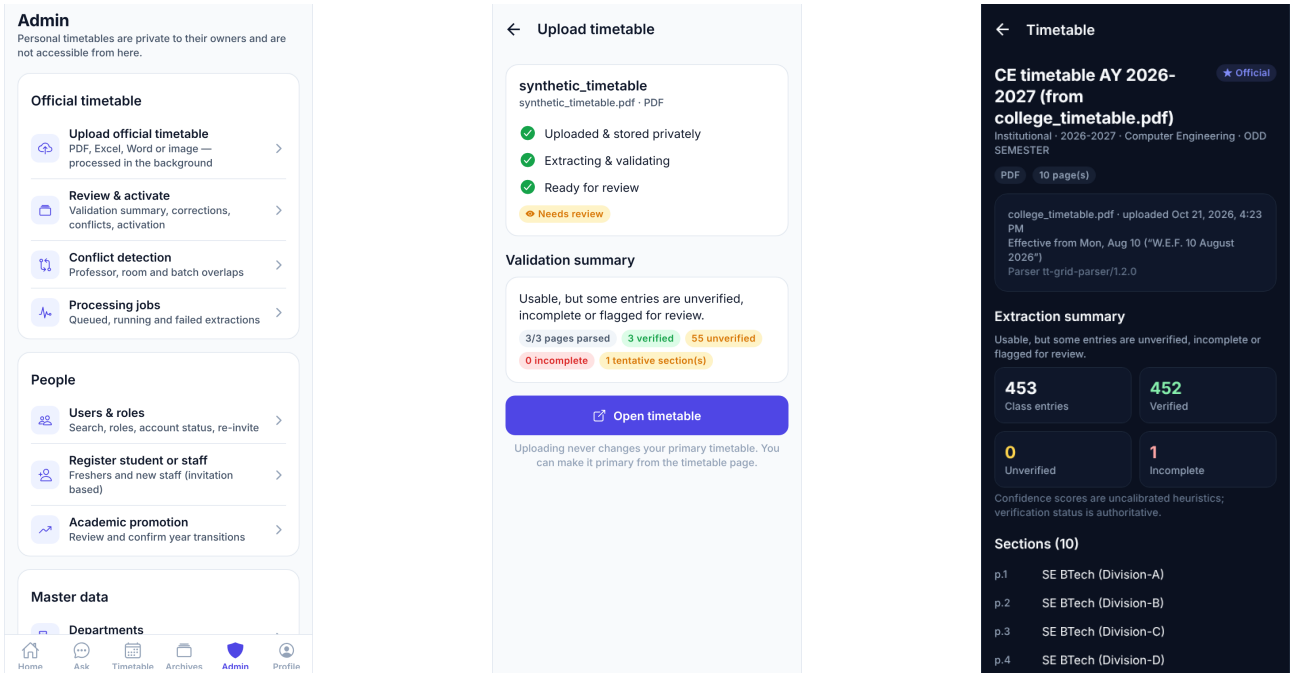
(a) Class details with source page and raw text (LLC: no teacher/room) **(b)** "Classes after lunch on Tuesday?" uses SE lunch **(c)** "Am I free at 2?" asks a.m. or p.m. **(d)** After choosing 2 PM and the date: answer

Figure 10.3: Natural-language search (Ask tab)



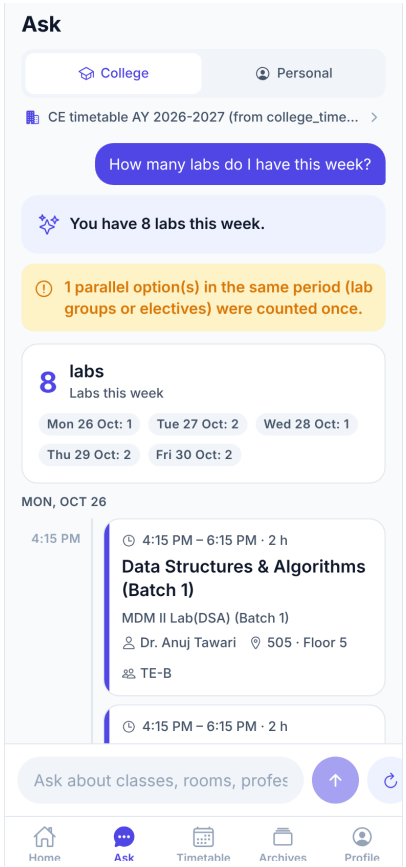
(a) B.E. combined electives (b) HOD home with conflict alert (c) The 2 genuine clashes (d) Personal archives (private)

Figure 10.4: Combined classes, HOD view, conflicts and personal domain

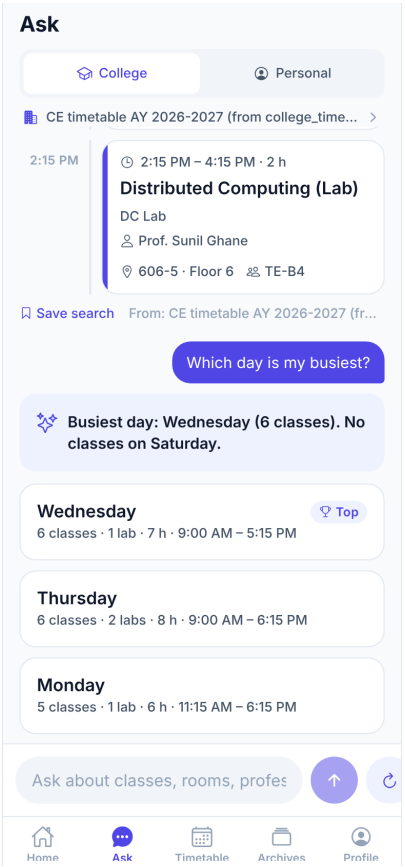


(a) Admin hub (b) Upload processed by worker (c) Real PDF: 453 entries, 452 verified, 10 sections

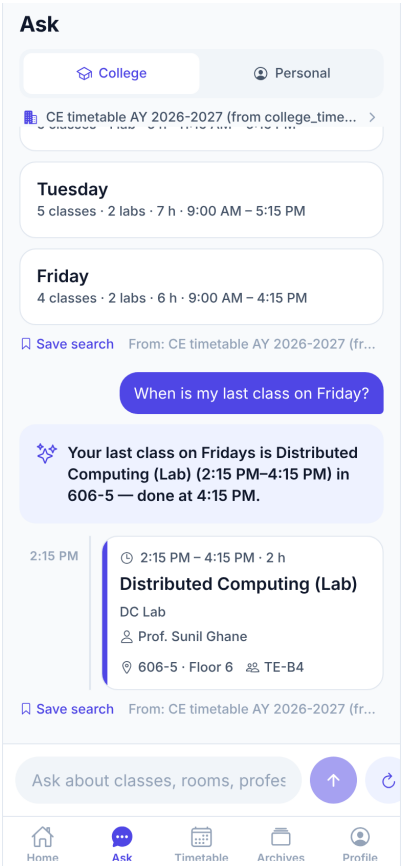
Figure 10.5: Administrator workflow



(a) "How many labs do I have this week?": count with per-day breakdown



(b) "Which day is my busiest?": day-wise summary



(c) "When is my last class on Friday?"

Figure 10.6: Composed query plans answered from the real timetable (TE-B4 student)

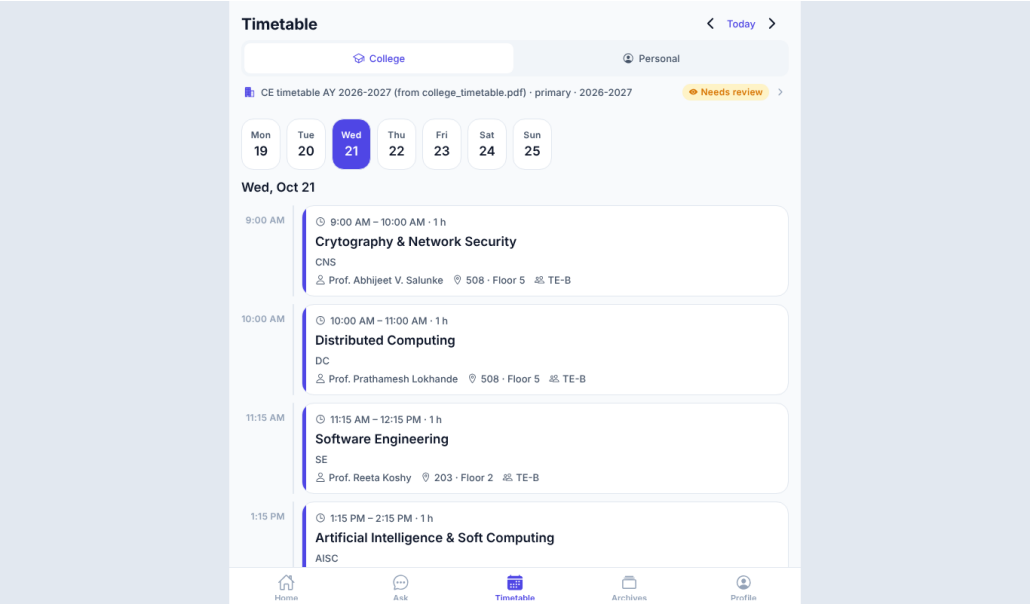


Figure 10.7: The same app in a laptop browser

10.3 Functional Results

- **Extraction (FR-04):** the real PDF (10 pages, 420 kB) produced 570 rows: 453 class entries, of which **452 verified** and 1 INCOMPLETE (M.Tech. HSS on Friday has no time label in the PDF), plus breaks and activities. All 10 pages were parsed, and 46 batches, 38 courses, 52 faculty and 42 rooms were recognised.
- **Conflicts (FR-13):** exactly two clashes, both confirmed by hand in the PDF: (1) *Friday 2:15–3:15*, Prof. Sushama Pande has *LA* for SE-B in 609 (page 2) and *Data Science Lab C3* for SE-C in 606-4 (page 3); (2) *Tuesday 2:15–3:15*, room **603-2** has B.E. *PE III – Gen AI* (page 9) and TE-D *DC Lab D2* (page 8).
- **Role views (FR-09/10):** SE-A1 sees only A1's labs on Wednesday; B.E. A and C both see the combined B.E. electives; LLC appears without teacher or room; the noon row 12:15–13:15 is a normal class.
- **Queries (FR-11):** "Do I have classes after lunch?" uses 14:15 for S.E. and 13:15 for T.E. (each from its own section). "Am I free at 2?" asks a.m./p.m. A student without a batch gets a setup message instead of an invented answer.
- **Privacy (FR-14):** another user's personal timetable returns 404, even for the Admin.

10.4 Performance Results

Summarised from Chapter 8: median search latency 37–47 ms for a single user; 46.5 req/s at 4 clients and about 55–60 req/s at saturation with 4 workers on a 4-core VM; 0 wrong answers in 960 requests at 32 clients; extraction of the 10-page PDF in 1.65 s using 82 MB peak memory; 12 MB database.

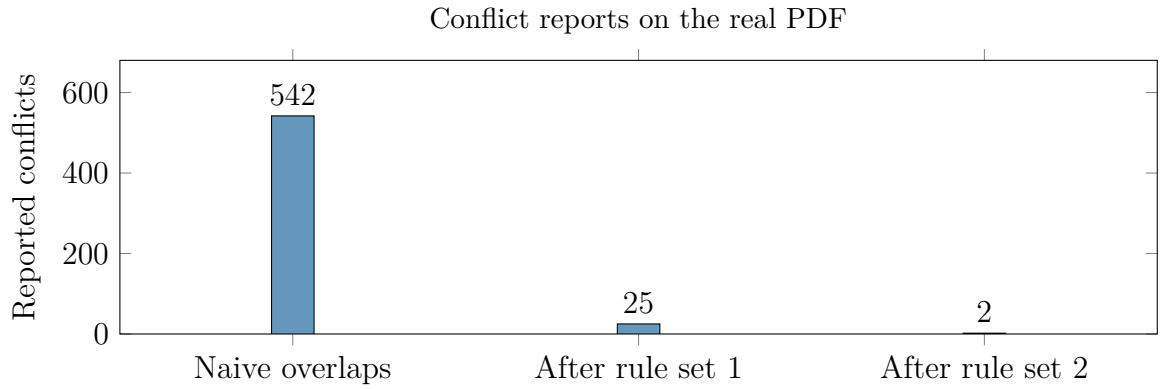


Figure 10.8: Effect of intended-overlap rules: from 542 raw overlaps to the 2 genuine clashes

10.5 Comparison with Existing Systems

Using the criteria of Table 2.2, the proposed system is the only one that reads the published timetable and also answers role-scoped questions safely. Table 10.2 compares the effort for common tasks. The manual times are our own estimates from trying each task with the PDF; they were not measured in a controlled study.

Table 10.2: Task effort: manual PDF lookup vs. the app (manual times are estimates)

Task	**Manual PDF**	**Dynamic Timetable**
Find my next class and its room	find page, row, column, lab group, legend: about 1 min	open app: 0 taps
Find a free room at 11 am on floor 6	check every page for every room: 10+ min	one question, <1 s
Check all teacher/room clashes	impractical by hand (453 classes)	automatic, on every upload
Know whether the timetable changed	compare two PDFs by eye	notification + archive
Lab-group view (A1 vs A2)	read the stacked cells carefully	filtered automatically

10.6 Achievement of Objectives

Table 10.3: Objectives mapped to evidence

Obj.	Evidence	Status
O1	99.8% verified on the real PDF; uncertain values flagged (TC-07, TC-09)	Achieved
O2	PDF, scanned PDF/photo (OCR), XLSX/XLS, CSV, DOCX/DOC through one pipeline (TC-04, TC-10)	Achieved
O3	Five roles, server-side policy, role tests (TC-03), dashboards (Fig. 10.2, 10.4)	Achieved
O4	Specialised operations plus composed query plans (counts, first/last, day-wise, lab/lecture), with clarification (TC-16–TC-19, TC-31–TC-34)	Achieved
O5	2 genuine clashes, 0 false reports (TC-21)	Achieved
O6	Separate tables; IDOR and isolation tests (TC-22); archives	Achieved
O7	p50 37–47 ms; 0 wrong answers at 32 clients (TC-28, TC-29)	Achieved (dev hardware)
O8	Android/iPhone via Expo Go, laptop browser, Render demo	Achieved; store builds pending
O9	174 + 20 + 45 e2e steps passing	Achieved

10.7 Discussion of Findings

- **Real data is different from specifications.** Most of the extraction work went into conventions found only in the real PDF: the mislabelled noon row, codes missing from legends, LLC slots and combined B.E. pages. Testing on the real document from early on was decisive.
- **Honest uncertainty builds trust.** Marking the one entry without a time as INCOMPLETE, instead of guessing, kept every automatic answer correct.
- **The source document itself had errors.** The two clashes are real errors in the published timetable, so the system already adds value for the timetable committee.
- **Unexpected observation:** a small connectivity check (`HEAD /`) made the whole production web app appear offline. This shows why end-to-end tests in a production-like configuration matter.
- **Rules beat a black box here, if they compose.** A fixed list of question types felt limiting (counts, "busiest day", "last lab on Friday" were refused). Making the rules

compose into a query plan removed most of that limit while keeping reproducible answers in about 40 ms, no API cost and no injection risk.

10.8 Practical Applicability

The department can use the system as it is for the current semester: the Admin uploads the official PDF, reviews the flagged entry, activates it, and students and faculty use the web link or the app. Before full-college use, the college should provide always-on hosting, e-mail for account activation, and store builds of the app (Section 11.5). The approach also transfers to other departments and institutes that publish grid-style timetables. Their layouts may need parser adjustments, which the review workflow and regression tests are designed for.

Chapter 11: Conclusion and Future Scope

11.1 Conclusion

The college timetable is consulted by everyone every day, yet it exists only as a dense PDF that has to be read by hand, with no search, no personal view, no version control and no clash checking. This mini project built *Dynamic Timetable*, a role-based timetable monitoring system that reads the timetable as it is actually published (PDF, scanned image, Excel or Word), extracts every class with its source page and a verification status, detects clashes, and serves students, professors, the HOD, the Principal and the Admin through one app on Android, iPhone and laptop. Users can ask everyday questions in plain English. A deterministic compositional planner turns each question into a typed query plan (operation, filters, lab/lecture, date or week, time window, ordering) and compiles it into parameterised queries, so it answers counts, first/last and day-wise questions as well as the everyday ones in about 40 ms, never generates SQL from text, and asks for clarification instead of guessing. On the college's real 10-page timetable the system verified 452 of 453 class entries automatically and found the two genuine clashes in the published document. The system is covered by 174 back-end tests, 20 app tests and end-to-end browser tests, and it runs as a free online demo.

11.2 Achievement of Objectives

- **Achieved:** O1 (99.8% verified), O2 (all formats through one pipeline), O3 (five roles, server-side), O4 (composed query plans plus specialised operations), O5 (0 false clash reports), O6 (strict domain separation and archives), O9 (all tests passing).
- **Achieved on development hardware:** O7 (latency and correctness under 32 concurrent clients). It still has to be confirmed on production hardware.
- **Partially achieved:** O8. The app runs on Android and iPhone through Expo Go and on laptops through the web and the online demo, but installable store builds (APK/TestFlight) have not yet been produced.

11.3 Major Engineering Contributions

1. A timetable-aware extraction pipeline with constrained backtracking for inconsistent time labels, stacked lab-group parsing, page-local legends and per-entry provenance.
2. A conflict detector that separates intended overlaps (parallel labs, electives, shared sessions) from genuine clashes in $O(n \log n + m)$.

3. A safe, compositional natural-language interface: questions become typed query plans (operation, filters, lab/lecture, date or week, time window, ordering) compiled into parameterised queries with timetable-scoped entity resolution and clarification instead of guessing, injection-proof by design and no longer limited to a fixed list of questions.
4. A secure multi-role architecture: Argon2id, rotating refresh tokens with reuse detection, database-level ownership constraints (composite FKs, partial unique indexes) and audited corrections.
5. A free, reproducible deployment: Docker image, infrastructure as code (`render.yaml`) and continuous deployment from Git.

11.4 Current Limitations

- **Technical:** extraction accuracy has not been measured against a hand-labelled ground truth; new timetable layouts may need parser rules; the rate limiter is per process; co-taught cells ("RRS+SB") are not linked to both teachers' personal schedules.
- **Functional:** the system does not generate timetables; questions outside the 30 intents are politely refused; teacher location is the *scheduled* room, not tracked.
- **Operational:** the free demo sleeps when idle and its database expires every 30 days; push notifications and e-mail are not yet active; room floors were derived from room numbers and must be checked by the Admin.

11.5 Future Enhancements

1. Measure precision/recall on hand-labelled pages and publish per-field accuracy.
2. College single sign-on (OIDC/Google Workspace) instead of separate passwords.
3. Store builds (Android APK / Play Store, iOS TestFlight) with push notifications for timetable changes and "class starting in 10 minutes" reminders.
4. "What changed?" diff between two official versions, sent to affected users only.
5. Bulk CSV import of students, staff and rooms; many-to-many teacher links for co-taught classes.
6. An optional LLM *suggester* that proposes an intent and parameters for unusual phrasings, which must still pass the same validation and templates (never free SQL).
7. Room-utilisation analytics for the HOD and Principal; exam-timetable support.

8. Capacity testing on production hardware; a shared rate-limit store and vocabulary caching for multi-instance deployment.

11.6 Lessons Learned

- **Test with real data early.** The real PDF broke assumptions that synthetic data never would.
- **Make uncertainty visible.** Statuses, raw text and page numbers turned possible silent errors into review tasks.
- **Security is a design decision.** Choosing templates over generated SQL removed a whole class of attacks; our own audit still found a critical bug (P0-1), which showed us the value of reviewing our own code adversarially.
- **Automated tests make change safe.** The SDK upgrade and the parser changes were made with confidence because 200+ tests guarded behaviour.
- **Project management:** risk-first ordering (extraction before UI), small increments, one commit per working increment, and documenting decisions (why, not only what) kept the project on schedule and easy to explain.

References

[1] A. Schaerf, "A survey of automated timetabling," *Artificial Intelligence Review*, vol. 13, no. 2, pp. 87–127, 1999.

[2] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *European Journal of Operational Research*, vol. 140, no. 2, pp. 266–280, 2002.

[3] R. Lewis, "A survey of metaheuristic-based techniques for university timetabling problems," *OR Spectrum*, vol. 30, no. 1, pp. 167–190, 2008.

[4] R. Zanibbi, D. Blostein and J. R. Cordy, "A survey of table recognition: Models, observations, transformations, and inferences," *International Journal on Document Analysis and Recognition*, vol. 7, no. 1, pp. 1–16, 2004.

[5] T. Kieninger and A. Dengel, "The T-Recs table recognition and analysis system," in *Document Analysis Systems: Theory and Practice (DAS'98)*, LNCS vol. 1655, Springer, 1999, pp. 255–270.

[6] M. Göbel, T. Hassan, E. Oro and G. Orsi, "ICDAR 2013 table competition," in *Proc. 12th International Conference on Document Analysis and Recognition (ICDAR)*, 2013, pp. 1449–1453.

[7] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[8] I. Androutsopoulos, G. D. Ritchie and P. Thanisch, "Natural language interfaces to databases – an introduction," *Natural Language Engineering*, vol. 1, no. 1, pp. 29–81, 1995.

[9] V. Zhong, C. Xiong and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," *arXiv preprint arXiv:1709.00103*, 2017.

[10] T. Yu *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 3911–3921.

[11] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[12] R. Pedro, D. Castro, P. Carreira and N. Santos, "From prompt injections to SQL injection attacks: How protected is your LLM-integrated web application?," *arXiv preprint arXiv:2308.01990*, 2023.

[13] J. F. Allen, “Maintaining knowledge about temporal intervals,” *Communications of the ACM*, vol. 26, no. 11, pp. 832–843, 1983.

[14] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, “Role-based access control models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[15] A. Biryukov, D. Dinu and D. Khovratovich, “Argon2: New generation of memory-hard functions for password hashing and other applications,” in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016, pp. 292–302.

[16] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[17] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[18] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.

[19] OWASP Foundation, “OWASP Top 10:2021,” 2021. [Online]. Available: https://owasp.org/Top10/

[20] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. New York: McGraw-Hill, 2019.

[21] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[22] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.

[23] I. Sommerville, *Software Engineering*, 10th ed. Boston: Pearson, 2016.

[24] L. Lalescu *et al.*, “FET – Free Timetabling Software.” [Online]. Available: https://lalescu.ro/liviu/fet/

[25] UniTime, “UniTime – Comprehensive University Timetabling System.” [Online]. Available: https://www.unitime.org/

[26] Applied Software Consultants, “aSc Timetables.” [Online]. Available: https://www.asctimetables.com/

[27] Tabula, “Tabula: Extract tables from PDFs.” [Online]. Available: https://tabula.technology/

[28] Camelot, "Camelot: PDF table extraction for humans." [Online]. Available: https://camelot-py.readthedocs.io/

[29] Artifex Software, "PyMuPDF documentation." [Online]. Available: https://pymupdf.readthedocs.io/

[30] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation – SELECT: The locking clause (FOR UPDATE, SKIP LOCKED)." [Online]. Available: https://www.postgresql.org/docs/16/sql-select.html

[31] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/

[32] Expo, "Expo documentation (SDK 58)." [Online]. Available: https://docs.expo.dev/

[33] Meta Platforms, "React Native documentation." [Online]. Available: https://reactnative.dev/

[34] Render, "Render documentation – Free instance types and free PostgreSQL." [Online]. Available: https://render.com/docs/free